CNN Architectures

Jefersson A. dos Santos
j.santos@sheffield.ac.uk

Road Map

Previous lectures:

- Introduction to Deep Learning
  - Basics
  - Training Process
- Convolutional Neural Networks
  - Convolutional Layers
  - Data Augmentation
  - Fine Tuning

This lecture:

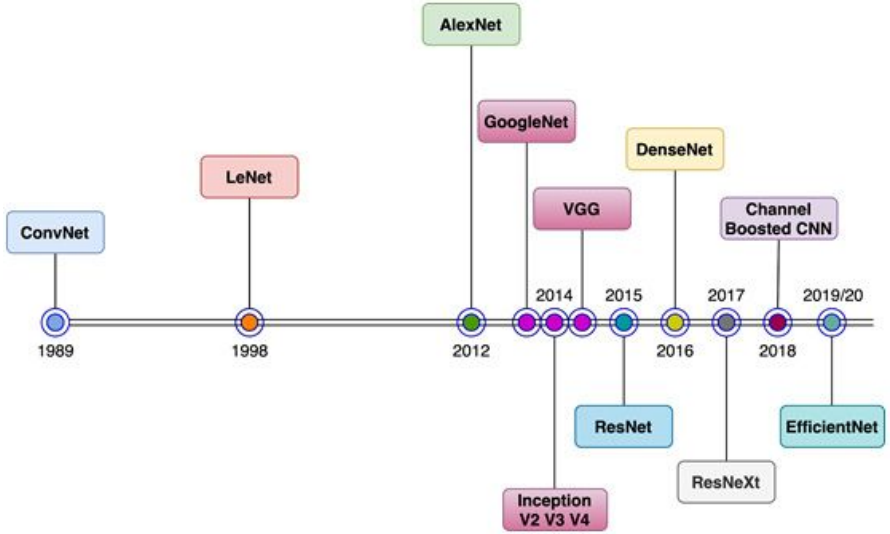
- Advanced Concepts
- CNN Architectures
  - VGG, ResNet, DenseNet, Inception

Next lectures:

- Object Detection
  - YOLO, R-CNN Family
- Semantic Segmentation
  - FCN, SegNet, U-Net

Influential CNN architectures

- LeNet [1] (1998)
- AlexNet [2] (2012)
- VGG [3] (2014)
- GoogLeNet [4] (2014)
- ResNets [5] (2015)
- DenseNets [6] (2016)
- EfficientNet [7] (2019)

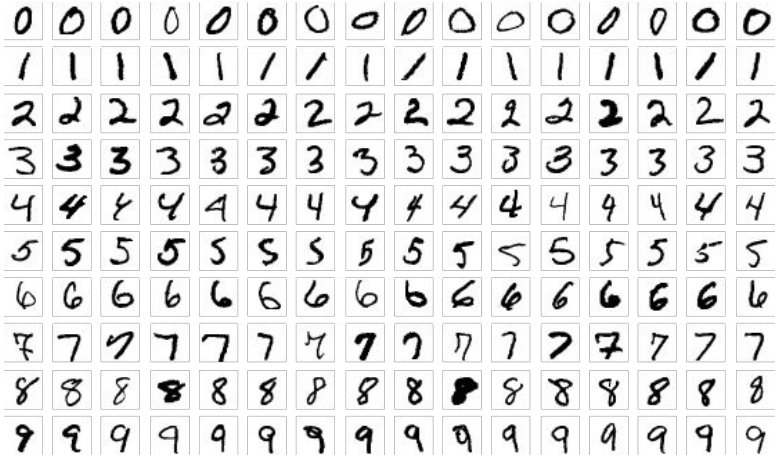


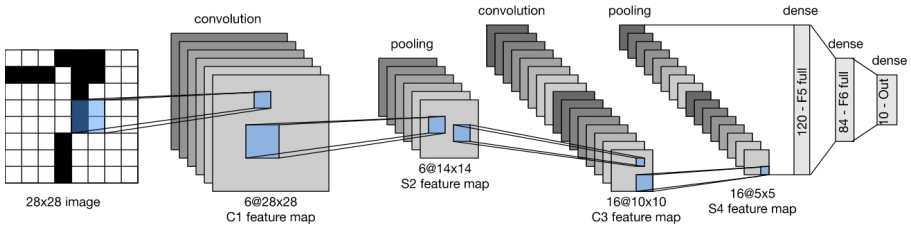
1. Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11):2278–2324, 1998.
2. Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In Advances in neural information processing systems, pages 1097–1105, 2012.
3. Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556, 2014.
4. Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 1–9, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778, 2016.
6. Gao Huang, Zhuang Liu, Kilian Q Weinberger, and Laurens van der Maaten. Densely connected convolutional networks. In Proceedings of the IEEE conference on computer vision and pattern recognition, volume 1, page 3, 2017.
7. Tan M, Le Q. Efficientnet: Rethinking model scaling for convolutional neural networks. In International Conference on Machine Learning 2019 May 24 (pp. 6105-6114).

LeNet (1998)

- Recognizing handwritten digits in images
- Represented the culmination of a decade of research developing the technology
  - 1989: First study to successfully train CNNs via backpropagation

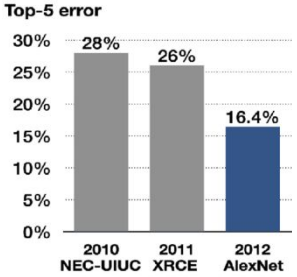
MNIST Dataset



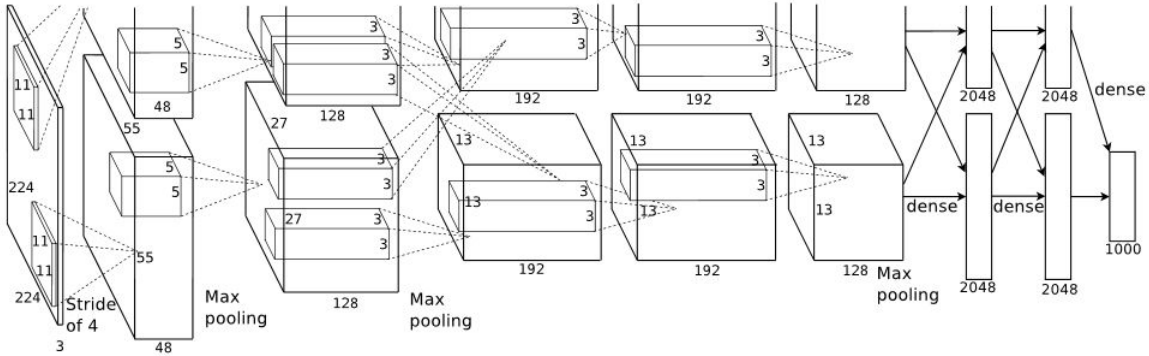


LeNet was adapted to recognize digits for **ATM deposits**. Some ATMs still use the original code from the 1990s by Yann LeCun and Leon Bottou!

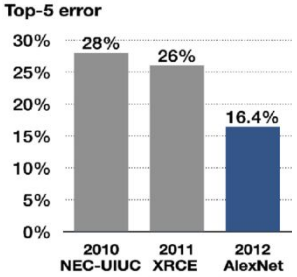
AlexNet (2012)



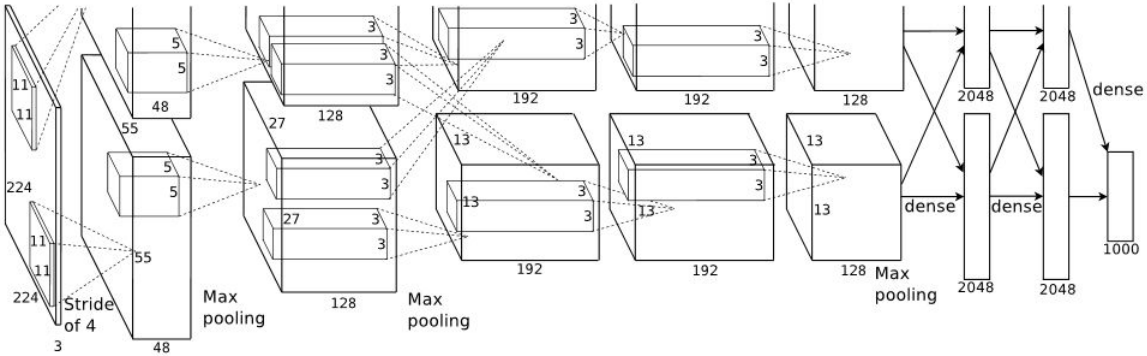
- First use of CNNs on challenging datasets
- ImageNet Large Scale Visual Recognition Competition (ILSVRC)
- Innovations:
  - New activation functions (ReLU)
  - GPU implementation
  - Regularization via Dropout



AlexNet (2012)



- First use of CNNs on challenging datasets
- ImageNet Large Scale Visual Recognition Competition (ILSVRC)
- Innovations:
  - New activation functions (ReLU)
  - GPU implementation
  - Regularization via Dropout



$$L(\boldsymbol{W}) = \frac{1}{N}\sum_i \underbrace{L_i(\boldsymbol{W}, \boldsymbol{x}_i, y_i)}_{\text{Data loss}} + \underbrace{\lambda\, R(\boldsymbol{W})}_{\text{Regularization}}$$

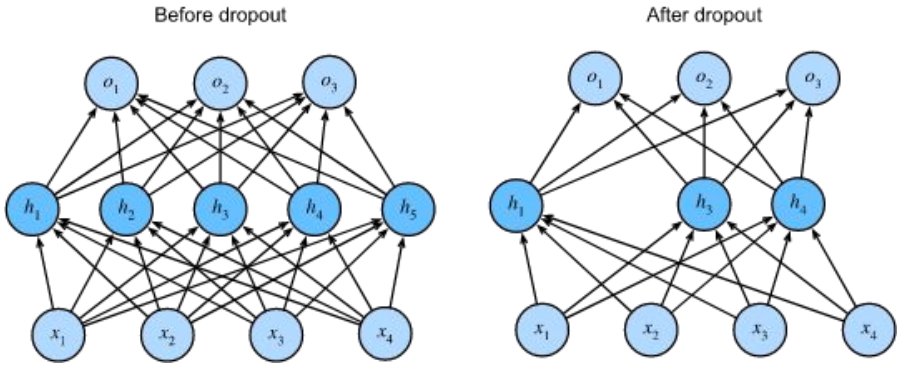
Data loss
match between
model and data

Regularization
model complexity

Regularization via Dropout

It involves randomly "dropping out" (deactivating) neurons during training to reduce co-dependency among them.

It helps in improving generalization by making the network less sensitive to specific patterns in the training data.



Recommended reading: http://d2l.ai/chapter_multilayer-perceptrons/dropout.html

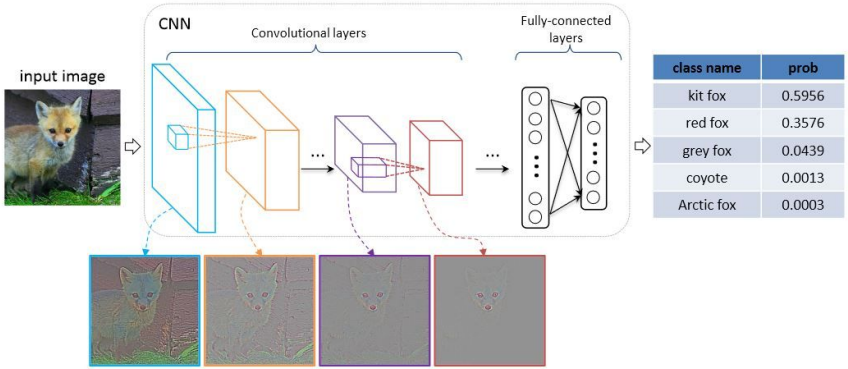
Regularization via Dropout

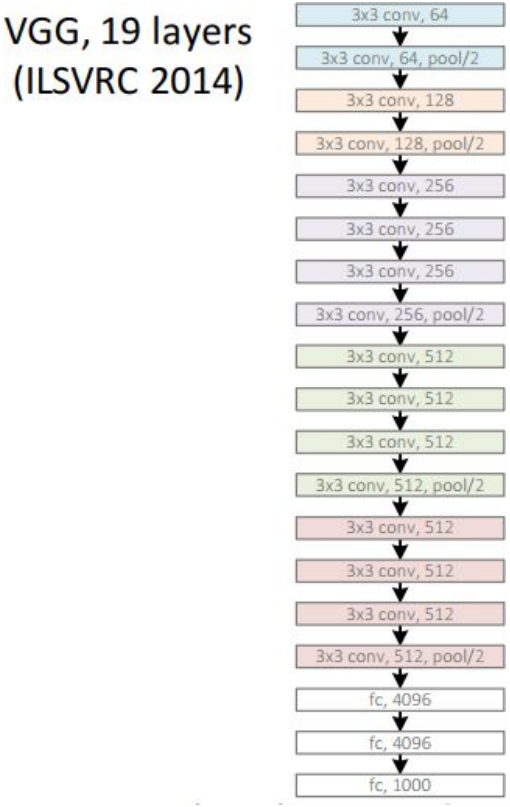
Implementation:

- Dropout is applied during training but not during testing or inference.
- Each training iteration randomly drops out a fraction of neurons, typically specified as a dropout rate (e.g., 0.2, meaning 20% dropout).
- Dropout can be implemented using dropout layers in deep learning frameworks like TensorFlow and PyTorch.

VGG (2014)

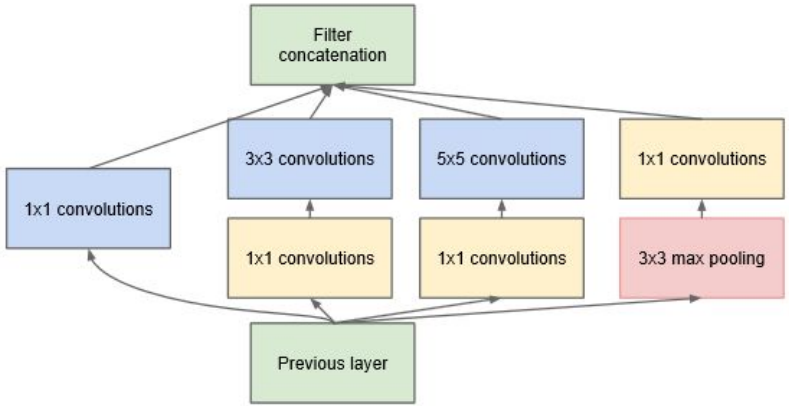
- Idea that deeper networks generate richer semantic representations
- 3x3 Convolutions (Padding 1) + Max-Pooling

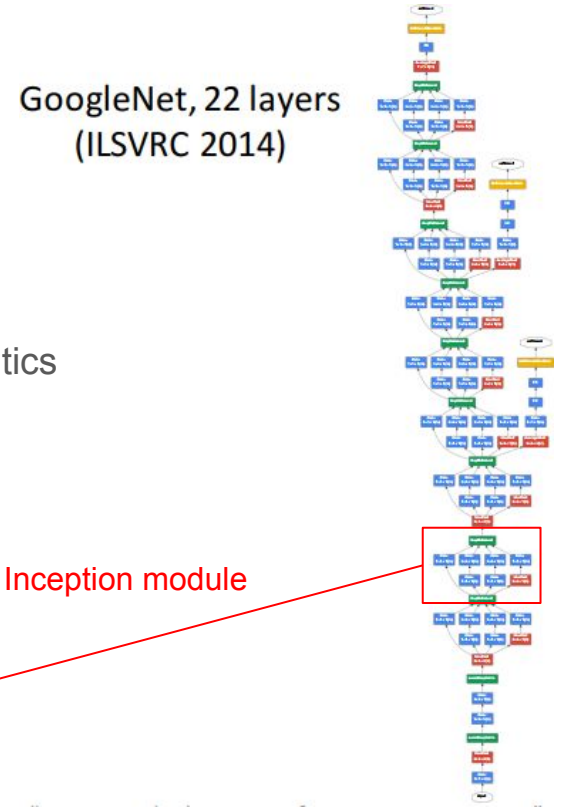




GoogLeNet (2015)

- Parallel convolution modules (Inception)
- Combine convolutions with different kernel sizes
- Idea:
  - deeper networks have representation highest level semantics



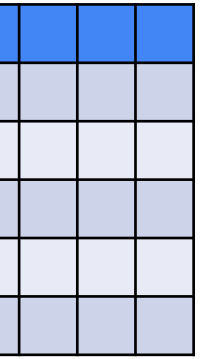


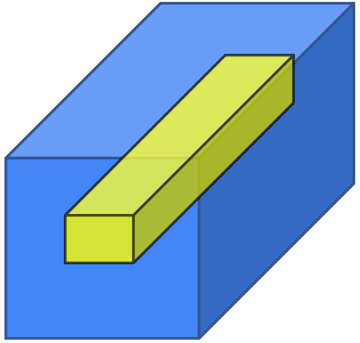
How does a 1x1 convolution work?

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

6×6

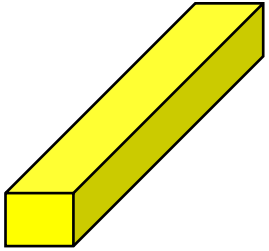
$*$ 2 $=$





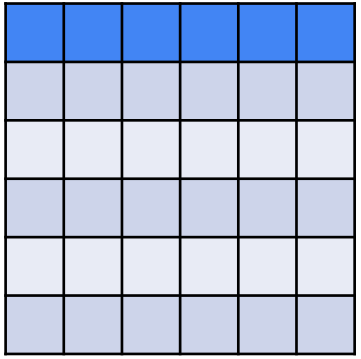
$6 \times 6 \times 32$

$*$



$1 \times 1 \times 32$

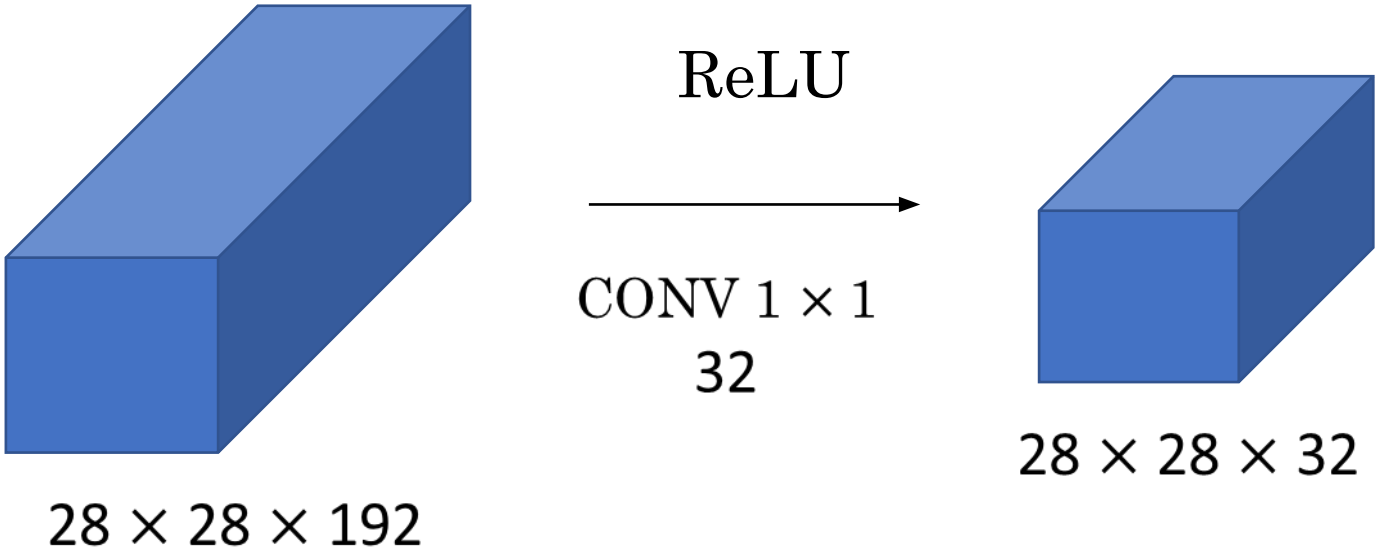
$=$



$6 \times 6 \times \#\ \text{filters}$

[Lin et al., 2013. Network in network]

How does a 1x1 convolution work?



[Lin et al., 2013. Network in network]

1x1 convolution work

It is used in many CNN architectures, such as the ResNet and Inception models.

A 1 x 1 convolution is useful when:

- We want to reduce the number of channels (feature transformation)
  - In the second example above, we reduced the input from 32 to 5 channels
  - Later we will see that by decreasing it we can save a lot of calculations
- If we have specified the number of 1 x 1 Conv filters to be the same as the input number of channels, the output will contain the same number of channels
  - Then Conv 1x1 will act as a nonlinearity and learn the nonlinearity operator.

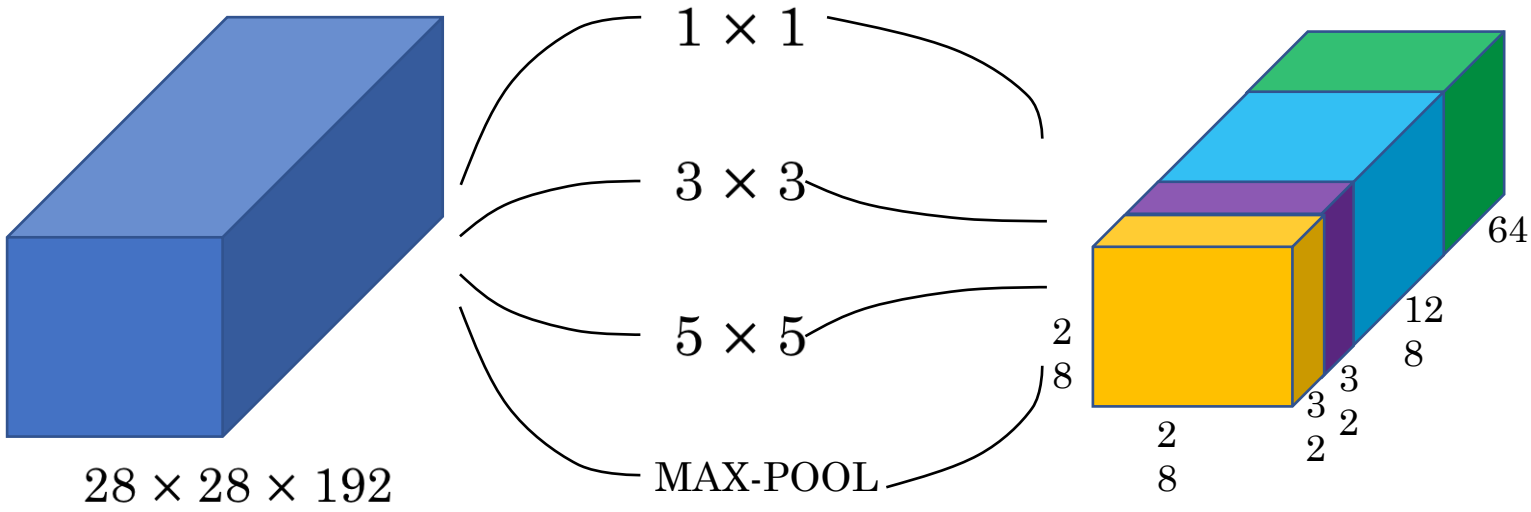
Motivation Behind Inception Layers

When you create a CNN, you have to decide on all the layers

- You will choose a 3 x 3 Conv or 5 x 5 Conv or perhaps a max pooling layer
- You have so many choices!

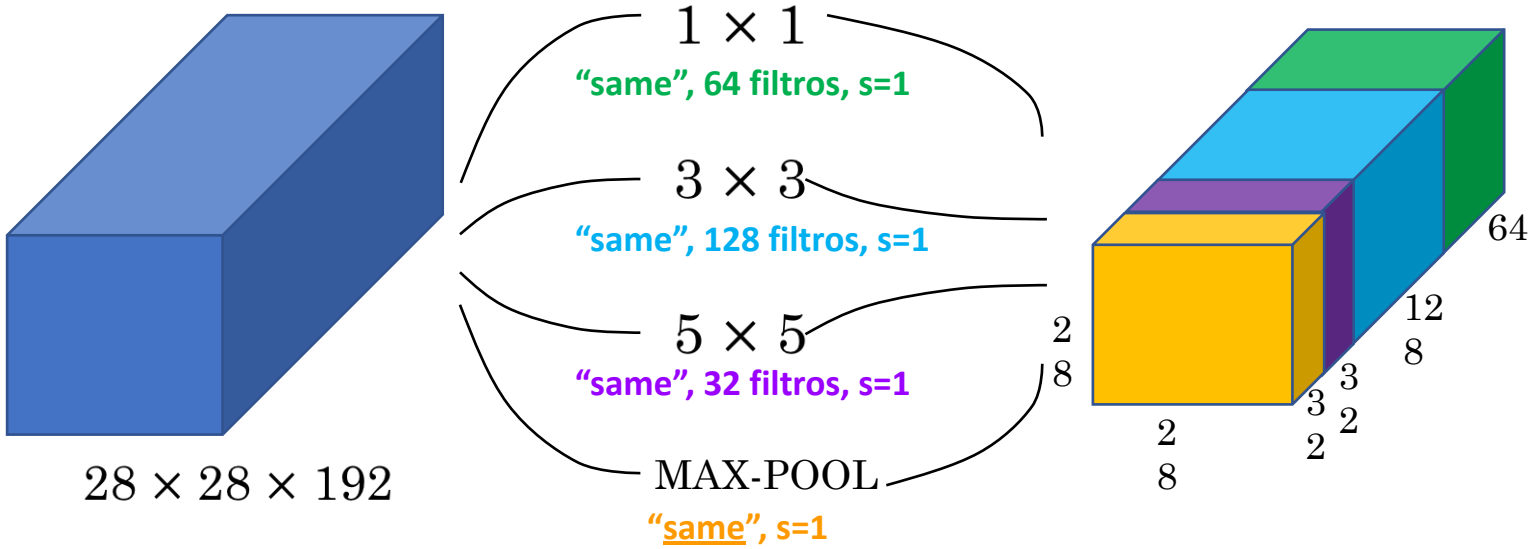
What inception tells us is: **why not use them all at once?**

Motivation Behind Inception Layers



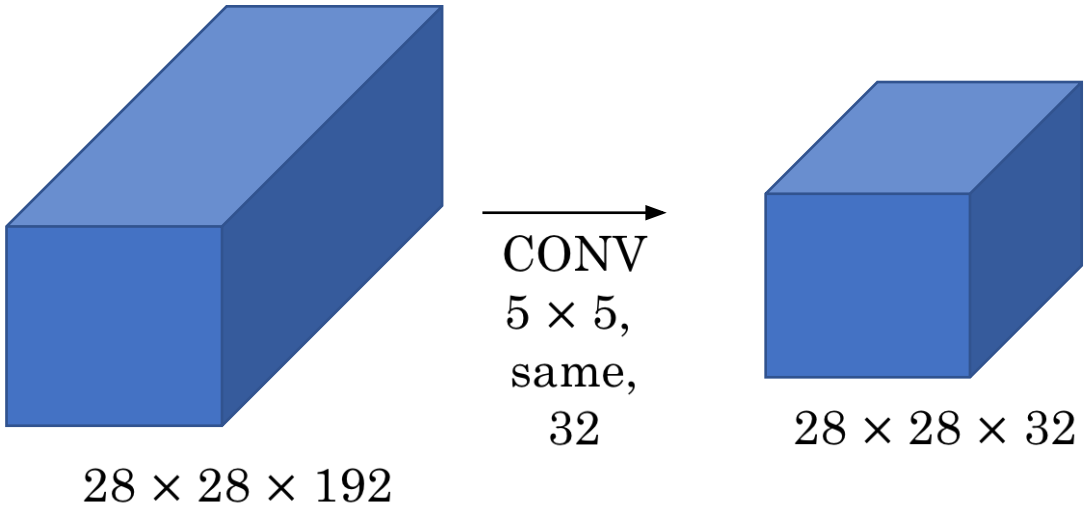
[Szegedy et al. 2014. Going deeper with convolutions]

Motivation Behind Inception Layers



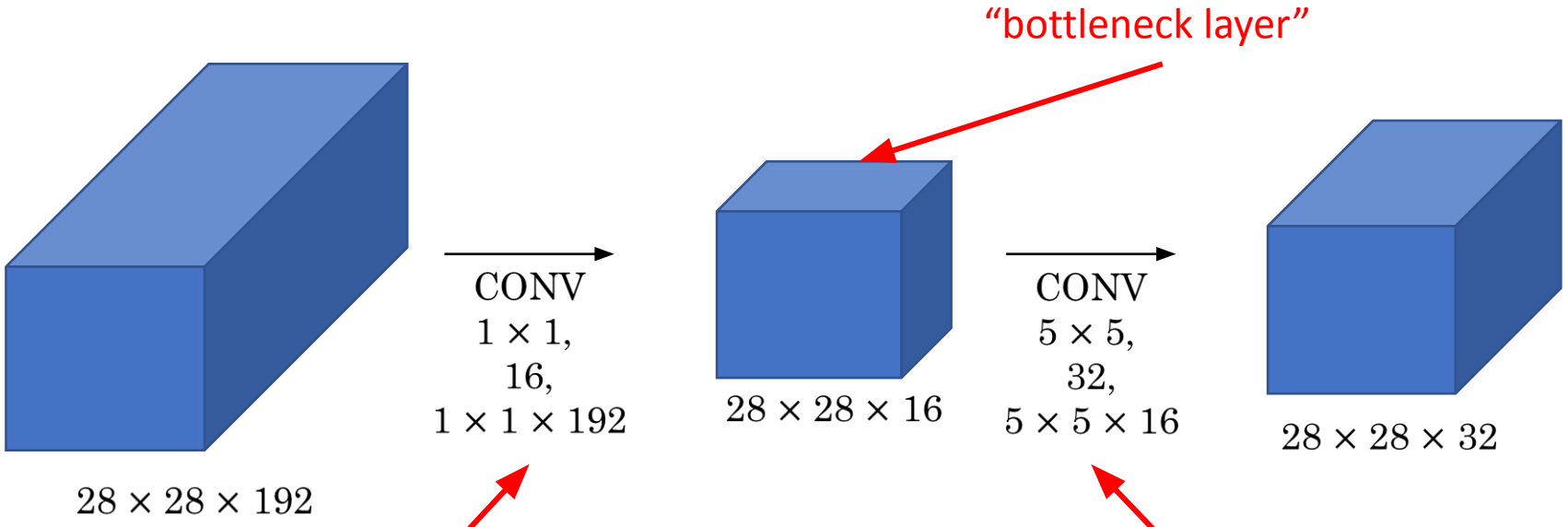
[Szegedy et al. 2014. Going deeper with convolutions]

Computational Challenge



Number of operations:
(28 * 28 * 32) * (5 * 5 * 192) = 120M!

Computational Challenge



Number of operations:
(28 * 28 * 16) * (1 * 1 * 192) = 2.4M

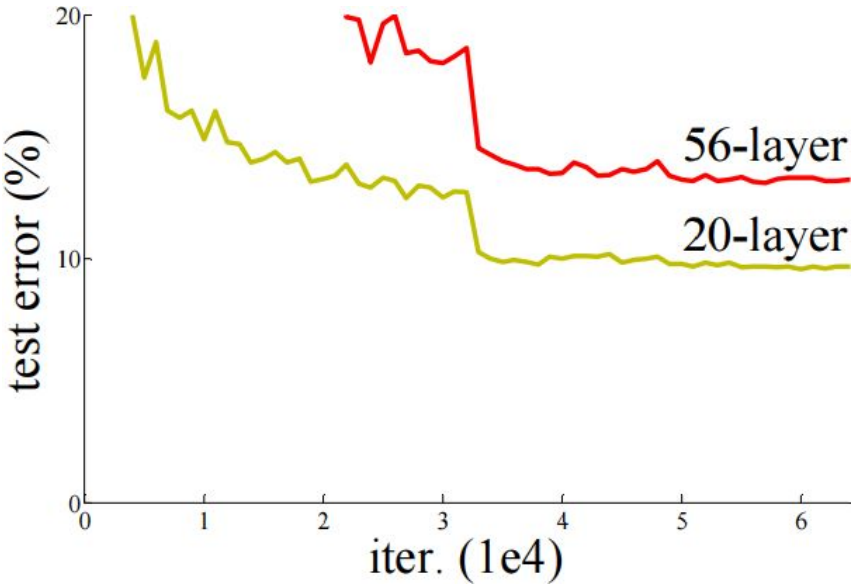
Number of operations:
(28 * 28 * 32) * (5 * 5 * 16) = 10M

12.4M << 120M

Semantic Level of Deep Neural Networks

VGG and GoogLeNet premise: more layers → higher semantic level information.

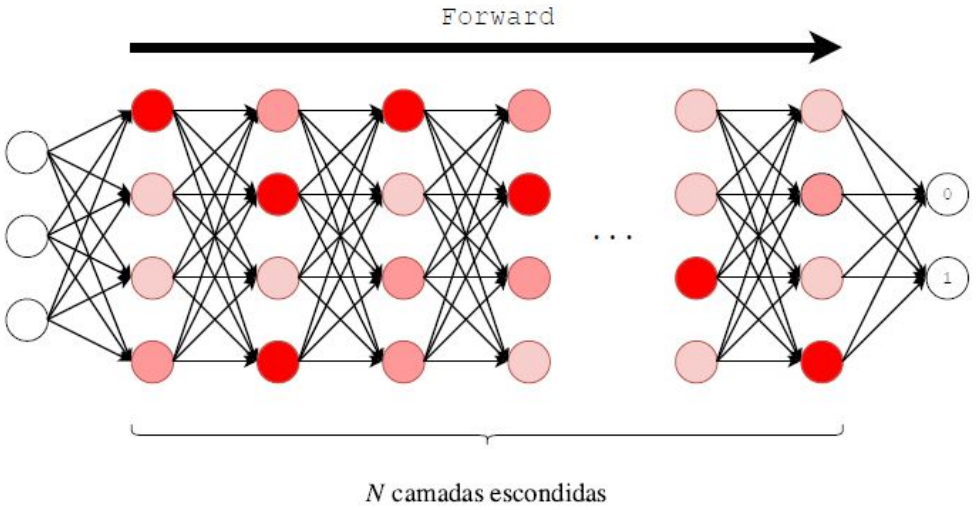
How to explain the following graph then?



Source: Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778, 2016.

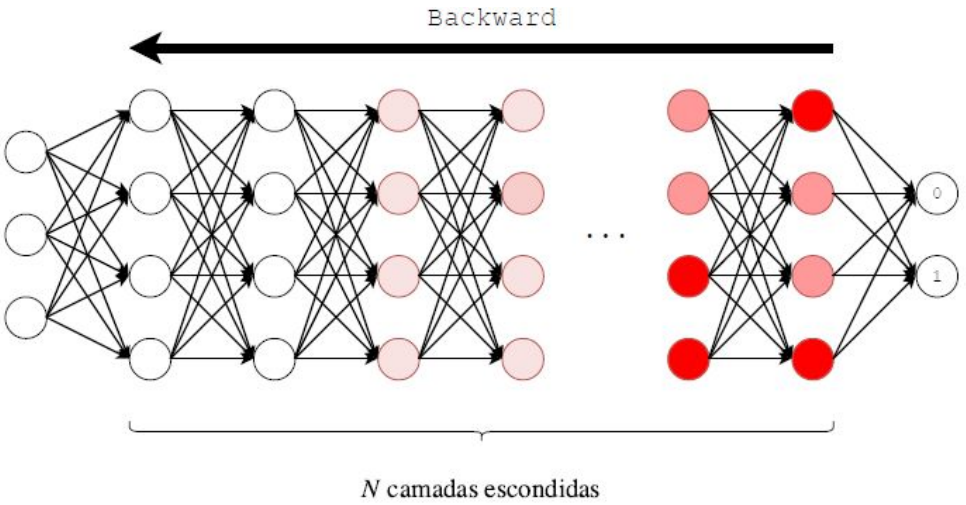
Vanishing Gradient

The gradients of the loss function with respect to the network's parameters become extremely small (close to zero) as they are backpropagated through the layers from the output to the input.



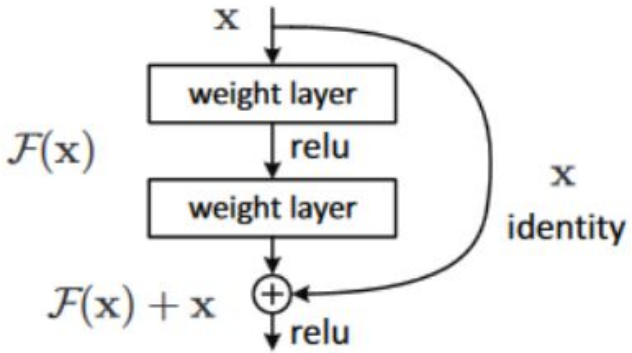
Vanishing Gradient

The gradients of the loss function with respect to the network's parameters become extremely small (close to zero) as they are backpropagated through the layers from the output to the input.



ResNets (2015)

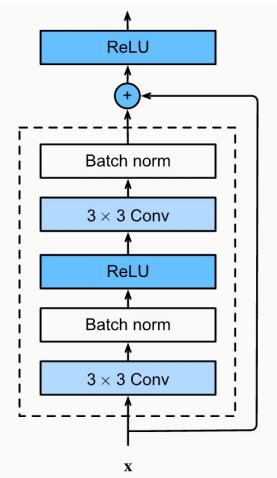
Deep Residual Networks try to overcome the Vanishing Gradient problem by using residual connections:



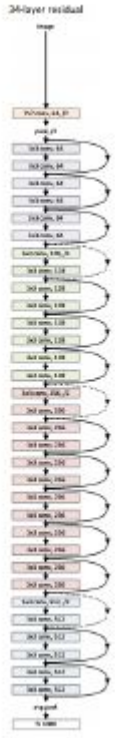
ResNets (2015)

Residual connections are a specific type of skip connection

Deep Residual Networks try to overcome the Vanishing Gradient problem by using residual connections:



VGG

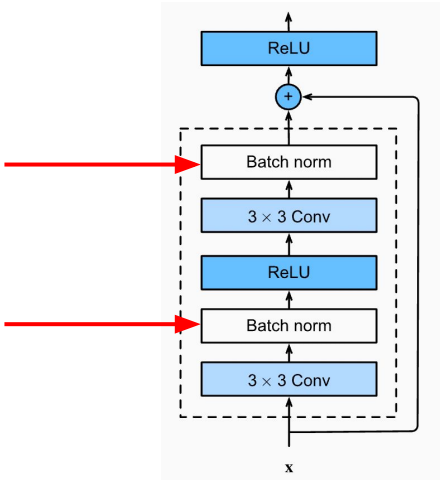


ResNet

ResNets (2015)

Residual connections are a specific type of skip connection

Deep Residual Networks try to overcome the Vanishing Gradient problem by using residual connections:

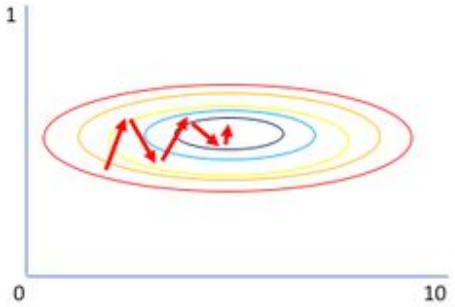


VGG

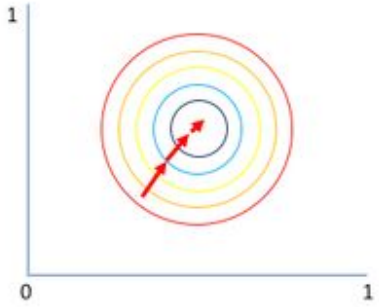
ResNet

Why normalizing data in machine learning is important?

- Improves convergence
- Prevents model bias
- Enhances model performance
- Stabilizes training
- Facilitates interpretation



Gradient of larger parameter dominates the update



Both parameters can be updated in equal proportions

Why normalizing data in machine learning is important?

- Improves convergence
- Prevents model bias
- Enhances model performance
- Stabilizes training
- Facilitates interpretation

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

standardization

$$x_{norm}^{(i)} = \frac{x^{(i)} - \mathbf{x}_{min}}{\mathbf{x}_{max} - \mathbf{x}_{min}}$$

min-max scaling ("normalization")

	input	standardized	normalized
0	0	-1.46385	0.0
1	1	-0.87831	0.2
2	2	-0.29277	0.4
3	3	0.29277	0.6
4	4	0.87831	0.8
5	5	1.46385	1.0

Batch Normalization

Technique used in deep learning to improve the training speed, stability, and performance of neural networks

Purpose: Normalizes the inputs of each layer by adjusting and scaling the activations, reducing internal covariate shift and improving convergence.

Benefits:

- Accelerates training by reducing the effects of vanishing/exploding gradients.
- Improves model stability by reducing sensitivity to weight initialization.
- Enhances generalization by acting as a regularizer during training.

Ioffe, Sergey, and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." *International conference on machine learning*. 2015.

Batch Normalization - Making the Activations Gaussian

Take a batch of activation at some layer. To make each dimension (k) look like a unit Gaussian, apply

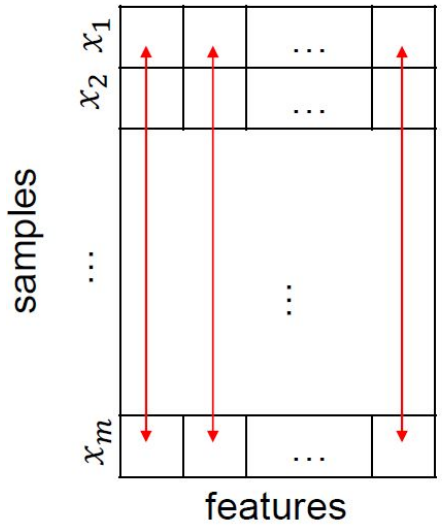
$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}\left[x^{(k)}\right]}{\sqrt{Var\left[x^{(k)}\right]}}$$

Note this function has a simple derivative

Batch Normalization - Making the Activations Gaussian

In practice, we take the empirical mean and variance.

For x over a mini-batch $\mathcal{B} = \{x_{1\ldots m}\}$:



$\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ computed for each dimension independently

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}$$

constant added for numerical stability

Batch Normalization - Making the Activations Gaussian

Normalize:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{Var[x^{(k)}]}}$$

and then allow the network to undo it, if "wished".

$$y^{(k)} = \gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)}$$

learnable parameters

Obs.: recall that (k) refers to a dimension.

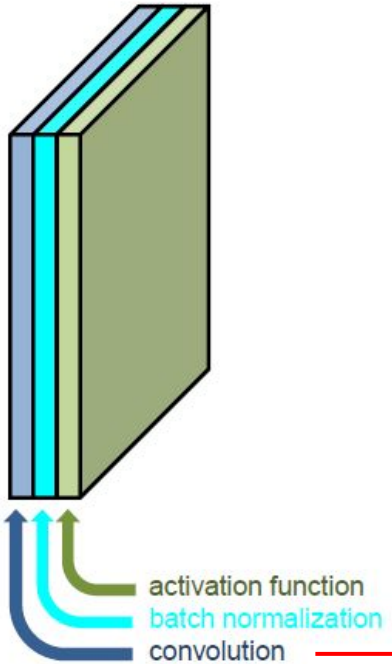
Notice that the network can learn:

$$\gamma^{(k)} = \sqrt{Var[x^{(k)}]}$$

$$\beta^{(k)} = \mathbb{E}[x^{(k)}]$$

to recover the identity mapping

ConvNet Layer with Batch Normalization



Batch normalization is inserted prior to the nonlinearity

$$y^{(k)} = \gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)}$$

We apply the operation on a per-channel basis *across all locations*

Recommended reading: https://d2l.ai/chapter_convolutional-modern/batch-norm.html

Batch Normalization Transform

Input: Values of x over a mini-batch $\mathcal{B} = \{x_{1\dots m}\}$;
Parameters to be learned: γ, β

Input:

$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i$ // mini-batch mean

$\sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$ // mini-batch variance

$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}$ // normalize

$y_i = \gamma\,\hat{x}_i + \beta \equiv BN_{\gamma,\beta}(x_i)$ // scale and shift

- Improves gradient flow through the network
- Allows higher learning rates
- Reduces the dependence on the initialization

Ioffe, S. and Szegedy , C., Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, 2015.

Batch Normalization Transform

Input: Values of x over a mini-batch $\mathcal{B} = \{x_{1\dots m}\}$;
Parameters to be learned: γ, β

Input:

$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i$ // mini-batch mean

$\sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$ // mini-batch variance

$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}$ // normalize

$y_i = \gamma\,\hat{x}_i + \beta \equiv BN_{\gamma,\beta}(x_i)$ // scale and shift

At test time BN functions differently:

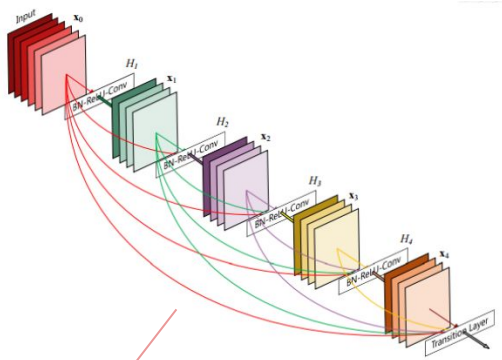
The mean/std are not computed based on the batch. Instead, fixed empirical values computed during training are used.

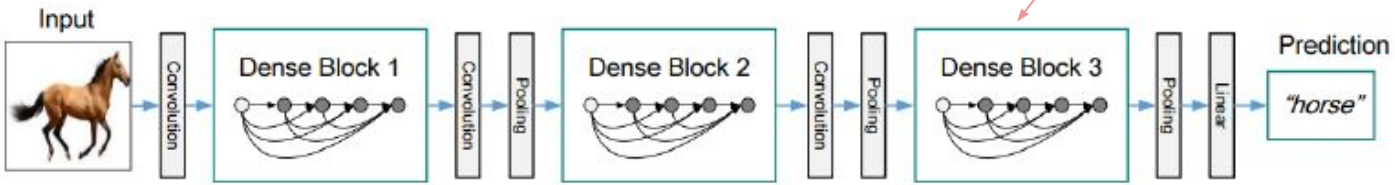
Ioffe, Sergey, and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." *International conference on machine learning*. 2015.

DenseNet (2016)

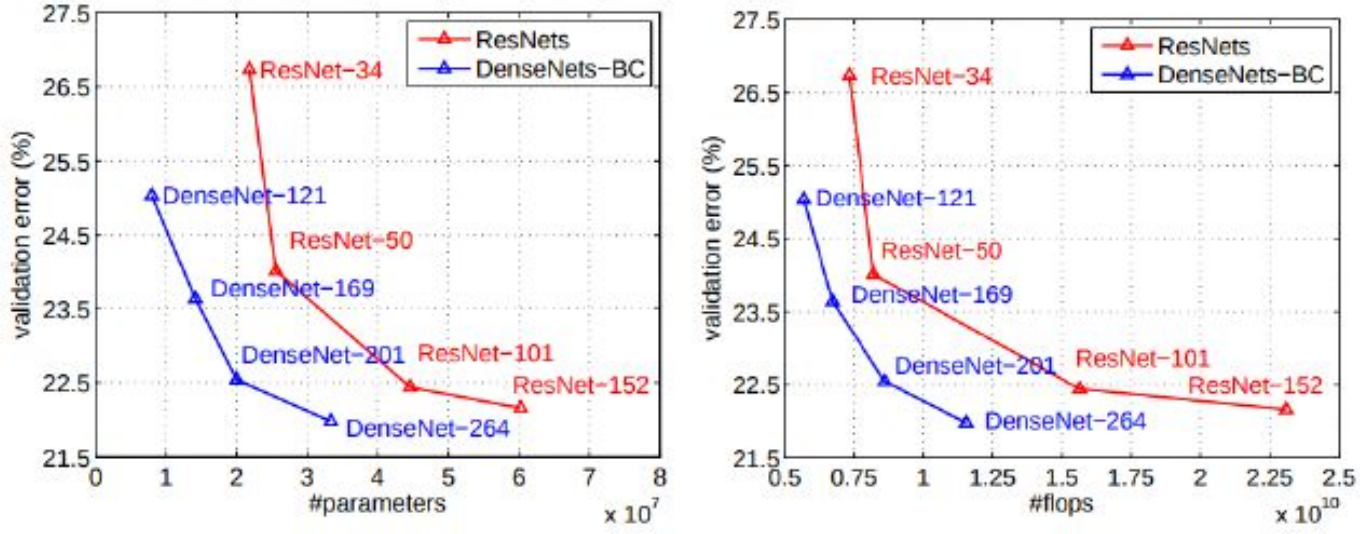
It was observed [5] that most of the ResNets filters do not contribute to the inference made by these networks.

Architectures with better parameter efficiency began to be proposed.





DenseNet (2016)



MobileNet and EfficientNet

Since DenseNets, the focus in the development of new CNN architectures has been on:

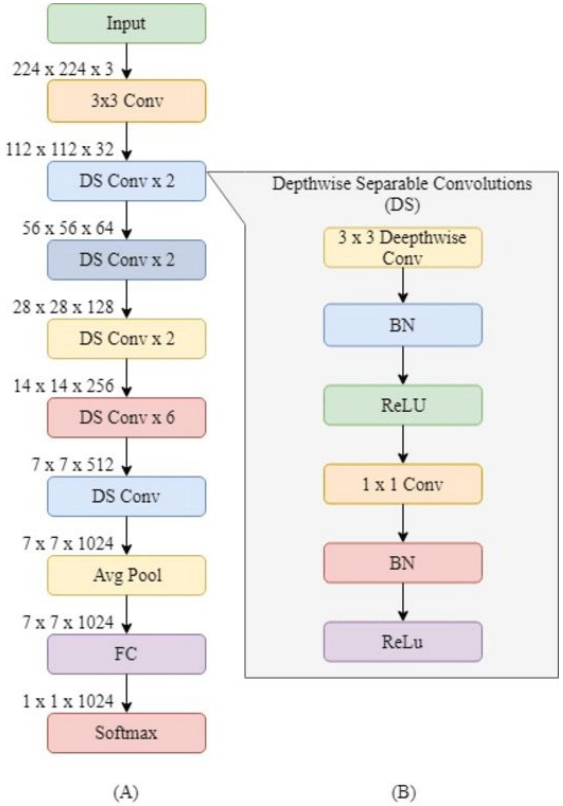
- Efficiency of parameters
- FLOPS
- Latency

Allow CNNs to be embedded in mobile devices and applications with severe limitations in computational power

MobileNet (2017)

MobileNet is a family of lightweight convolutional neural network (CNN) architectures designed for efficient and mobile-friendly deep learning applications.

- Depth-wise Separable Convolutions
- Depthwise Convolution
  - Apply a single filter per input channel, reducing the computational cost
- Pointwise Convolution (1x1 convs)
- Depthwise Separable Building Blocks
  - batch normalization + nonlinear activation functions
- Width Multiplier and Resolution Multiplier



EfficientNet (2019)

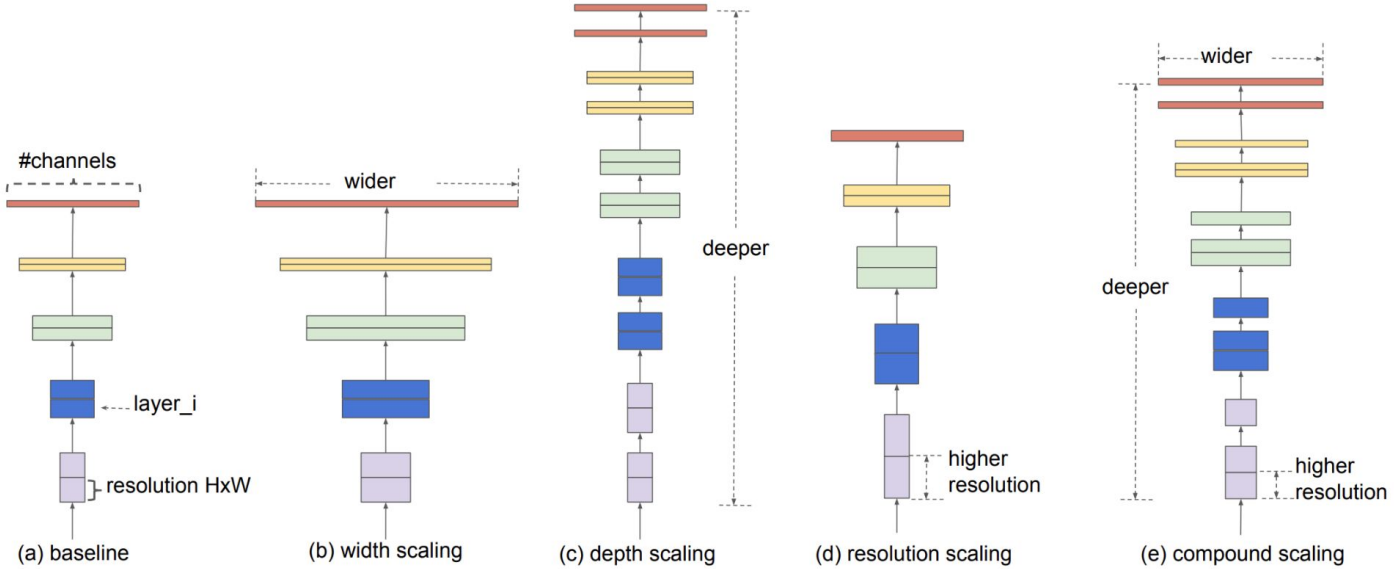
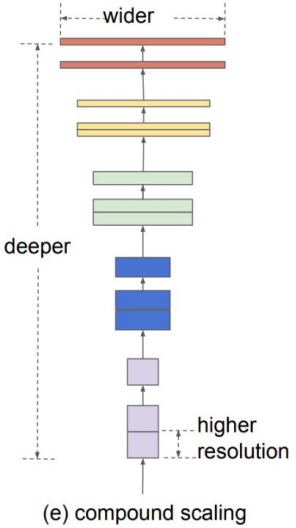


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

EfficientNet (2019)

Several innovations in the design and architecture of CNNs:

- Compound Scaling
  - scaling method that scales the network depth, width, and resolution simultaneously
- Efficient Blocks
- MBConv Blocks
- Feature Pyramid Network (FPN)
- Swish Activation Function

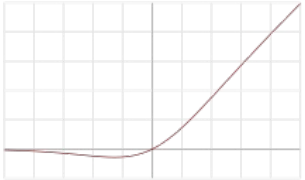


$$\text{Depth } d = \alpha^{\emptyset}, \text{Width } w = \beta^{\emptyset}, \text{Resolution } r = \gamma^{\emptyset}, (1)$$

$$\text{such that } \alpha.\beta^2.\gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

$$\text{swish}(x) = x\,\text{sigmoid}(\beta x) = \frac{x}{1+e^{-\beta x}}.$$



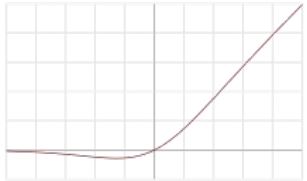
The network is fine-tuned for obtaining maximum accuracy but is also penalized if the network is very computationally heavy

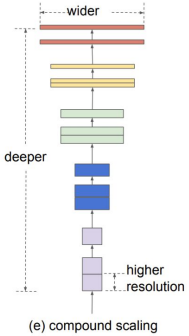
EfficientNet (2019)

Several innovations in the design and architecture of CNNs:

- Compound Scaling
  - scaling method that scales the network depth, width, and resolution simultaneously
- Efficient Blocks
- MBConv Blocks
- Feature Pyramid Network (FPN)
- Swish Activation Function

$$\text{swish}(x) = x\,\text{sigmoid}(\beta x) = \frac{x}{1+e^{-\beta x}}.$$



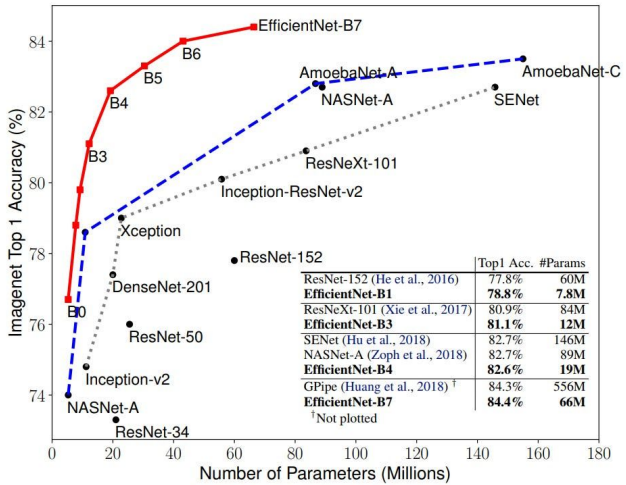


$$\text{Depth } d = \alpha^{\emptyset}, \text{Width } w = \beta^{\emptyset}, \text{Resolution } r = \gamma^{\emptyset}, (1)$$

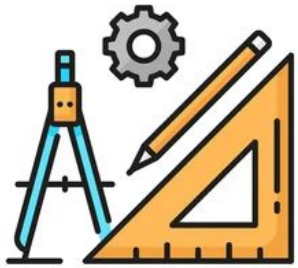
$$\text{such that } \alpha.\beta^2.\gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

The network is fine-tuned for obtaining maximum accuracy but is also penalized if the network is very computationally heavy



Quiz about CNN Architectures



https://www.menti.com/alxbmpqku4iv

Results:
https://www.mentimeter.com/app/presentation/alfp9qgk8uaafa38ahzsd6o8wtyov1ud

Deep-Based Object Detection - Part 1

Jefersson A. dos Santos
j.santos@sheffield.ac.uk

Road Map

Previous lectures:

- Introduction to Deep Learning
  - Basics
  - Training Process
- Convolutional Neural Networks
  - Convolutional Layers
  - Data Augmentation
  - Fine Tuning
- Modern CNN Architectures
  - VGG, ResNet, DenseNet, etc

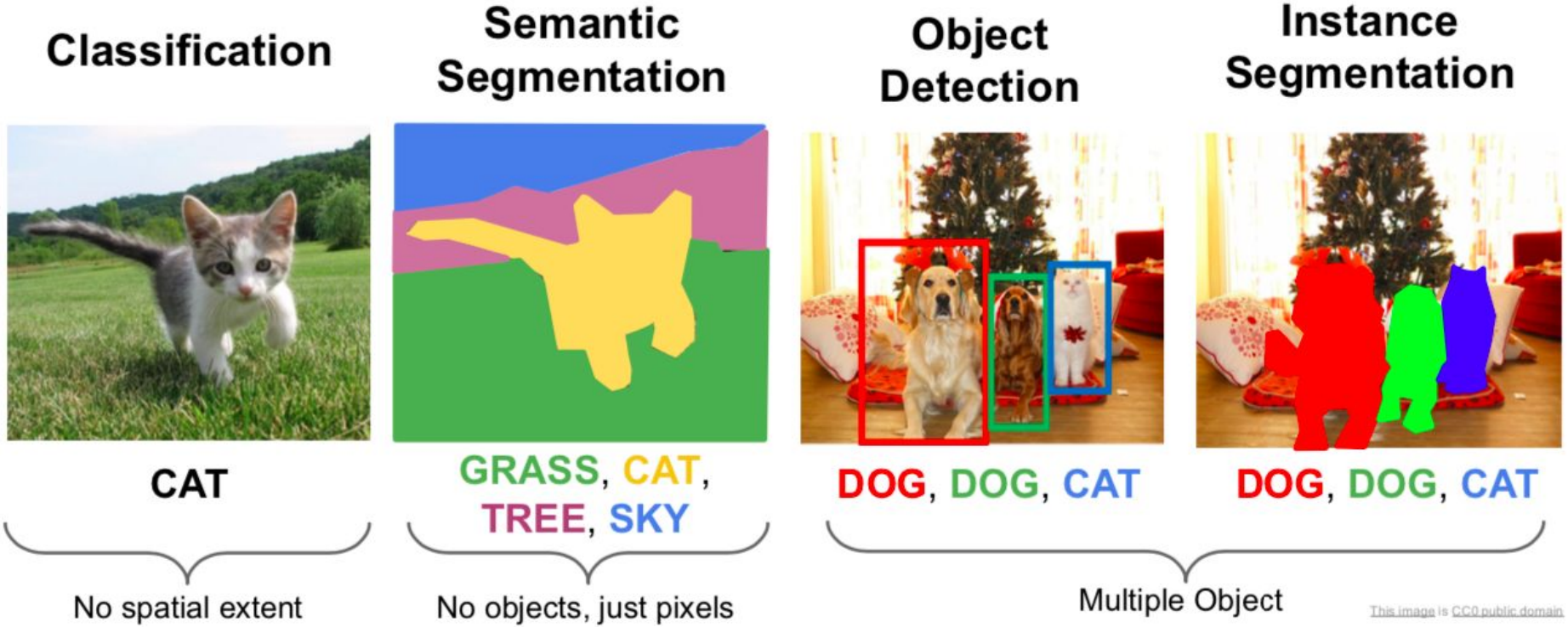
This lecture:

- Object Detection
- Single-Stage Architectures
  - YOLO

Next lectures:

- Double-Stage Architectures
  - R-CNN Family
- Semantic Segmentation

Computer Vision Tasks



Computer Vision Tasks

- **Classification:** returns the class associated with the image
- **Semantic Segmentation:** associates each pixel with a class; does not distinguish between instances of the same class
- **Object Detection:** returns class and location of each object in the image
- **Instance Segmentation:** assigns each pixel to an instance and classifies the instance

Before addressing object detection, let's consider only ONE object:

Classification with Localization:

- returns the class associated with the image and the location of the (single) object

Ideas for a simpler task are relevant for a more complex task

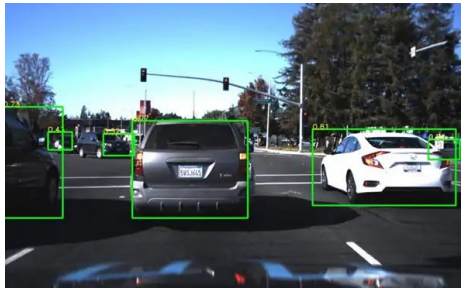
Object Detection

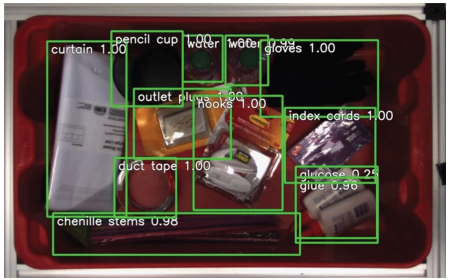
Task in computer vision that has developed a lot in the recent years.

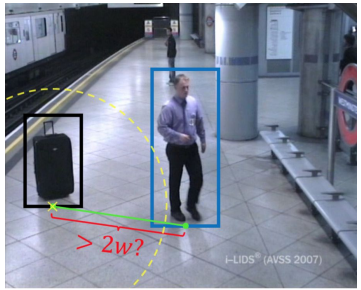
Applications:

- Autonomous vehicles:
  - planning routes by identifying the location of pedestrians, vehicles, streets, and obstacles in captured images.
- Robots (similar tasks).
- Security systems:
  - identifying anomalies (e.g., intruders or bombs).
  -

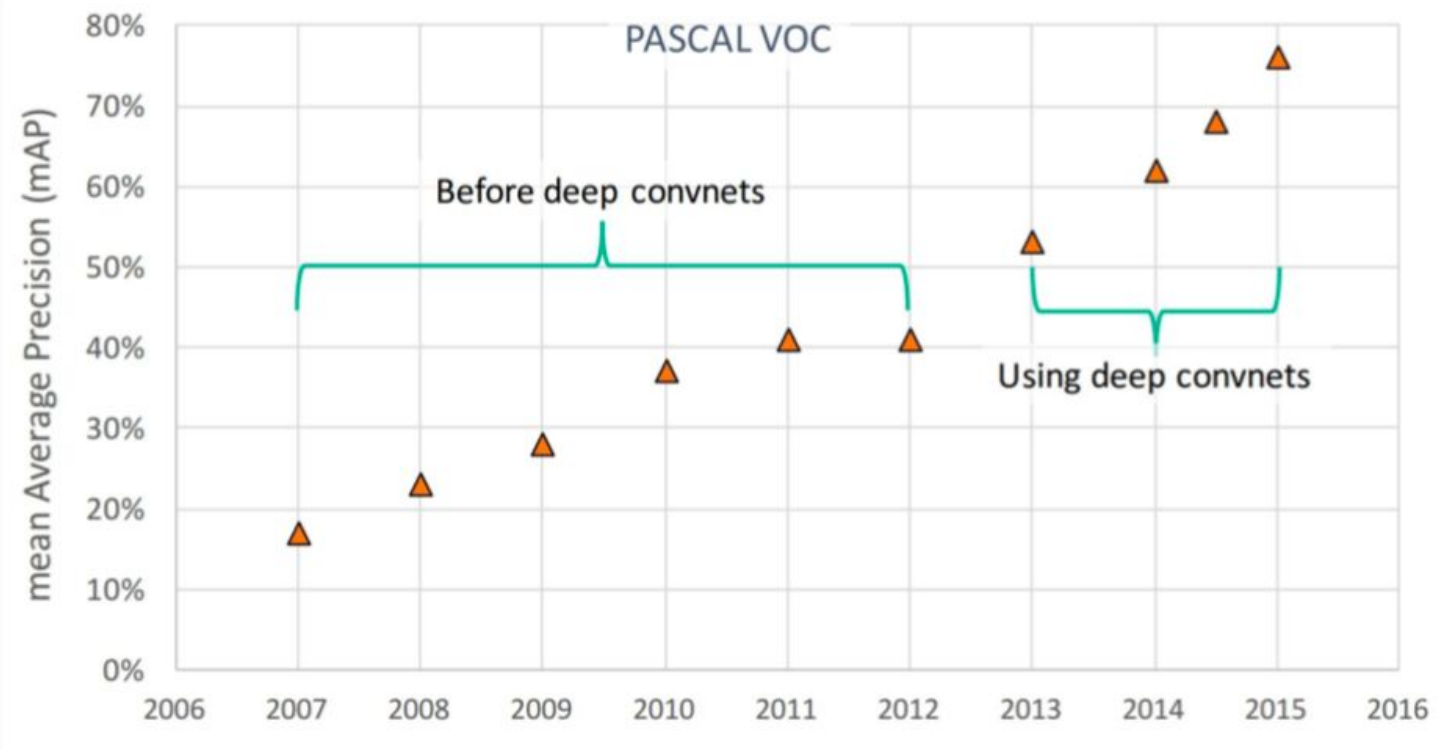
Performance in these tasks is now MUCH better than ten years ago.







Deep Learning Impact



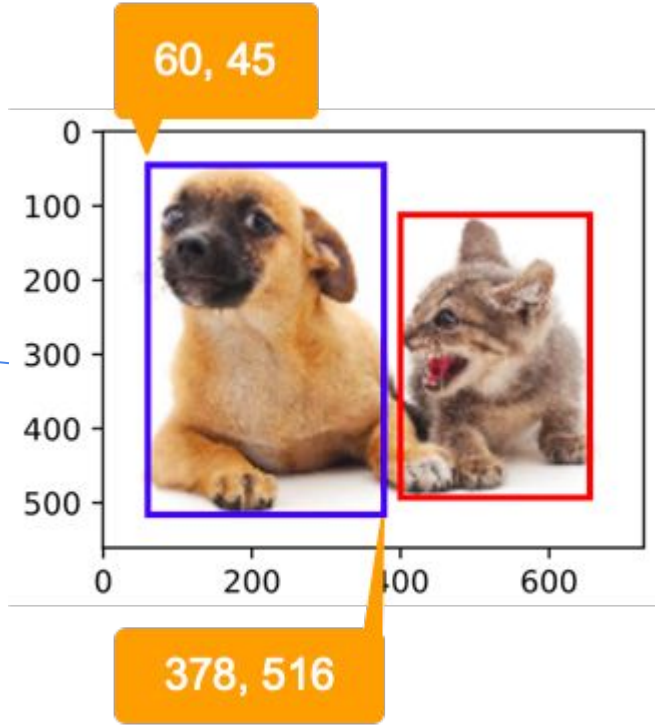
Credits: Ross Girshick, 2015

Bounding Box

A bounding box is defined by 4 numbers:

- top left x, top left y, bottom right x, bottom right y
- top left x, top left y, width, height
- center x, center y, width, height

Example with this convention

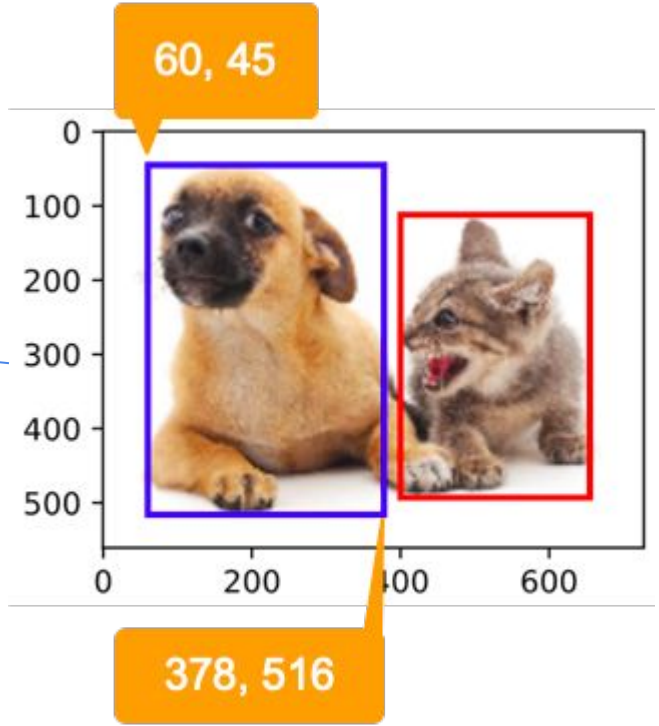


Bounding Box

A bounding box is defined by 4 numbers:

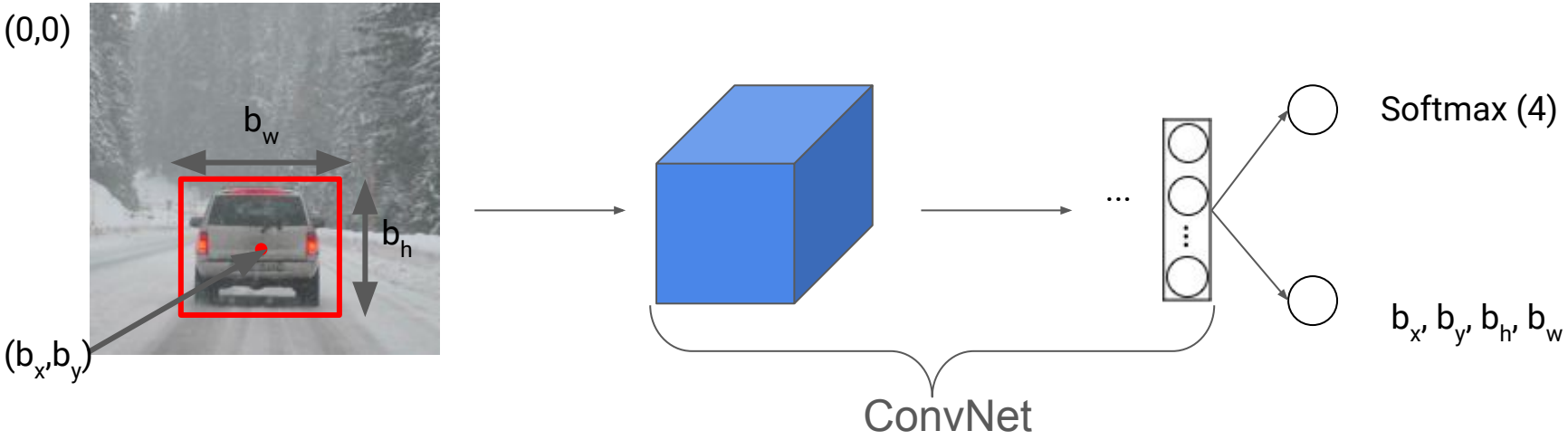
- top left x, top left y, bottom right x, bottom right y
- top left x, top left y, width, height
- center x, center y, width, height

Example with this convention



For the rest of slides, we will use this convention, but first we will normalize the image scale

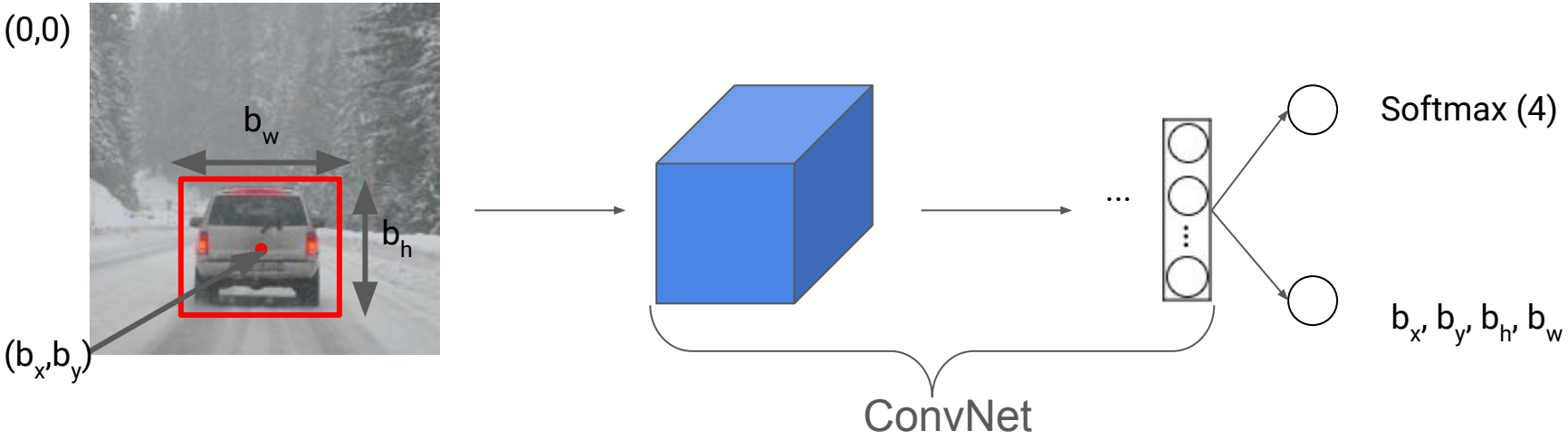
Example: classification with localization



Classes

1. Pedestrian
2. Car
3. Motorcycle
4. Background

Example: classification with localization



Classes

1. Pedestrian
2. Car
3. Motorcycle
4. Background

Bounding box

$b_x = 0.5$
$b_y = 0.7$
$b_h = 0.3$
$b_w = 0.4$

Treats location as a regression problem

Classification with localization

- Convention:
  - Image: top left corner: (0,0); bottom right: (1,1)
- Image Classification:
  - Convolutional network, last layer is FC with outputs to softmax (4)
- With localization:
  - Adds 4 new outputs that locate bounding box: b_x, b_y, b_h, b_w

    Center (b_x,b_y), width b_w, height b_h

Defining the y label

Need to retur: b_x, b_y, b_h, b_w, class label (1-4)

x=

Classes

1. Pedestrian
2. Car
3. Motorcycle
4. Background

Defining the y label

Need to retur: b_x, b_y, b_h, b_w, class label (1-4)

x=

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$ → Is this an object?

$$\begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix} \qquad \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$ → Don't care

Classes

1. Pedestrian
2. Car
3. Motorcycle
4. Background

Possible loss function (with quadratic error):

$$L(\hat{y}, y) = \begin{cases} (\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 + \ldots (\hat{y}_8 - y_8)^2 & \text{if } y_1 = 1 \\ (\hat{y}_1 - y_1)^2 & \text{if } y_1 = 0 \end{cases}$$

Defining the y label

- Vector y is composed by 8 coordinates:
  - p_c: is this an object?
  - b_x, b_y, b_h, b_w: bounding box
  - c_1, c_2, c_3: class
- Two cases:
  - Is an objeto, p_c=1, compute the loss in all other coordinates
  - No object, p_c=0, we "don't care" about other coordinates when computing the loss
  - In practice, error is measured with a different function in each coordinate:

    p_c (logistic regression loss), bounding box (squared error), class (cross-entropy loss)

55

Car detection example

Training set:

x	y
	1
	1
	1
	0
	0

Car detection example

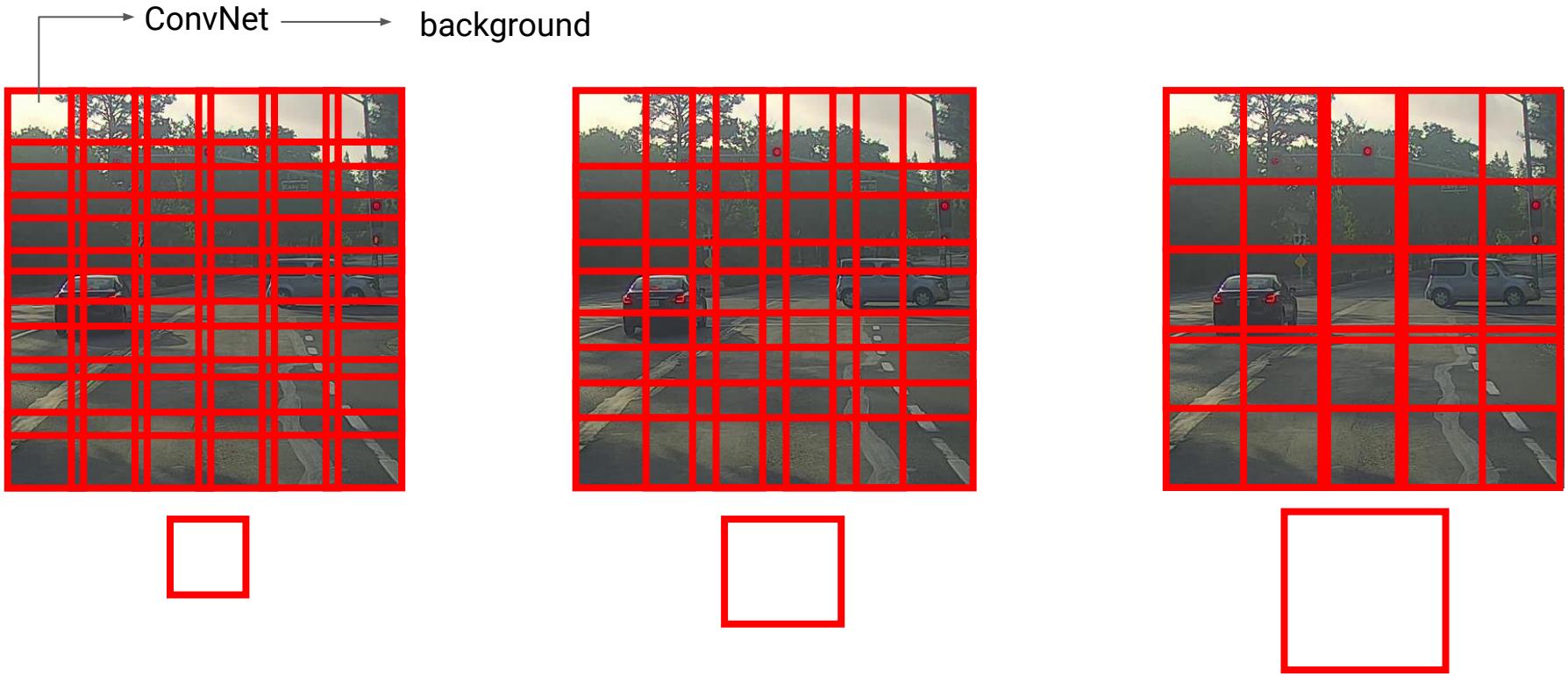
Training set:

x	y
	1
	1
	1
	0
	0

Suppose we have a classification network:

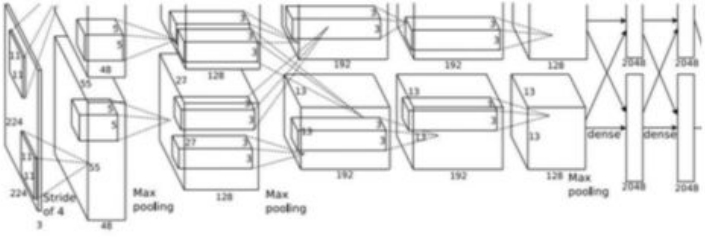
We can use sliding windows!

Sliding windows detection



Object detection: multiple objects

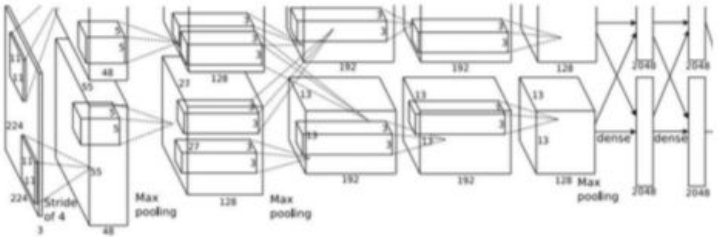
- Apply a CNN to different parts of the image
- CNN classifies each section as an object or background



Dog? NO
Cat? NO
Background? YES

Object detection: multiple objects

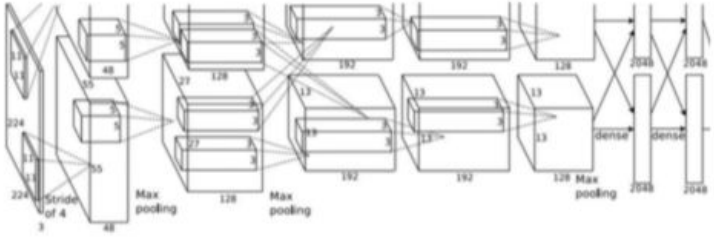
- Apply a CNN to different parts of the image
- CNN classifies each section as an object or background



Dog? YES
Cat? NO
Background? NO

Detecção de objetos: múltiplos objetos

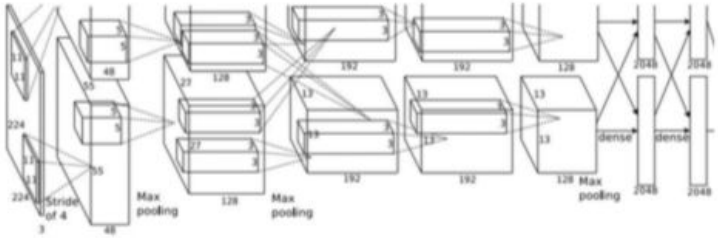
- Apply a CNN to different parts of the image
- CNN classifies each section as an object or background



Dog? YES
Cat? NO
Background? NO

Detecção de objetos: múltiplos objetos

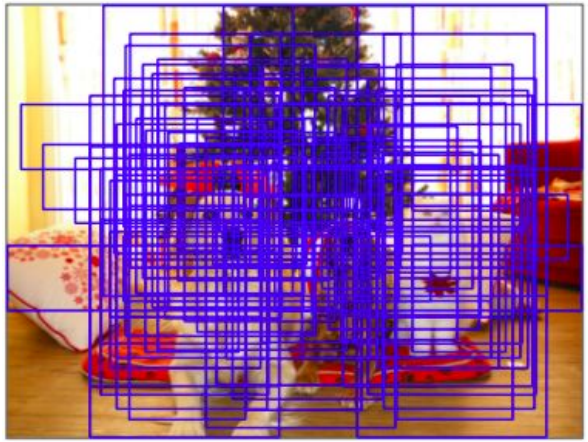
- Apply a CNN to different parts of the image
- CNN classifies each section as an object or background

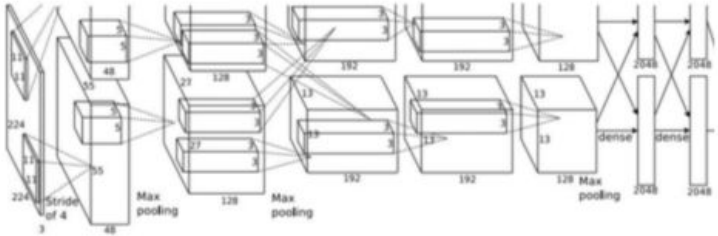


Dog? NO
Cat? YES
Background? NO

Detecção de objetos: múltiplos objetos

- Apply a CNN to different parts of the image
- CNN classifies each section as an object or background





Dog? NO
Cat? YES
Background? NO

Problem: Need to apply CNN to huge number of locations, scales, and aspect ratios, very computationally expensive!

Detection by sliding windows - Summary

- It starts with a small window that slides from left to right, top to bottom, according to the chosen stride. For each window, we use ConvNet to return a prediction.
- The same algorithm starts again, but with larger windows.
- Problem: computational cost
  - We can increase stride, but performance may drop
- Alternative solutions:
  - not classify all regions (region proposal)
  - convolutional implementation of sliding windows → YOLO!

YOLO Detection

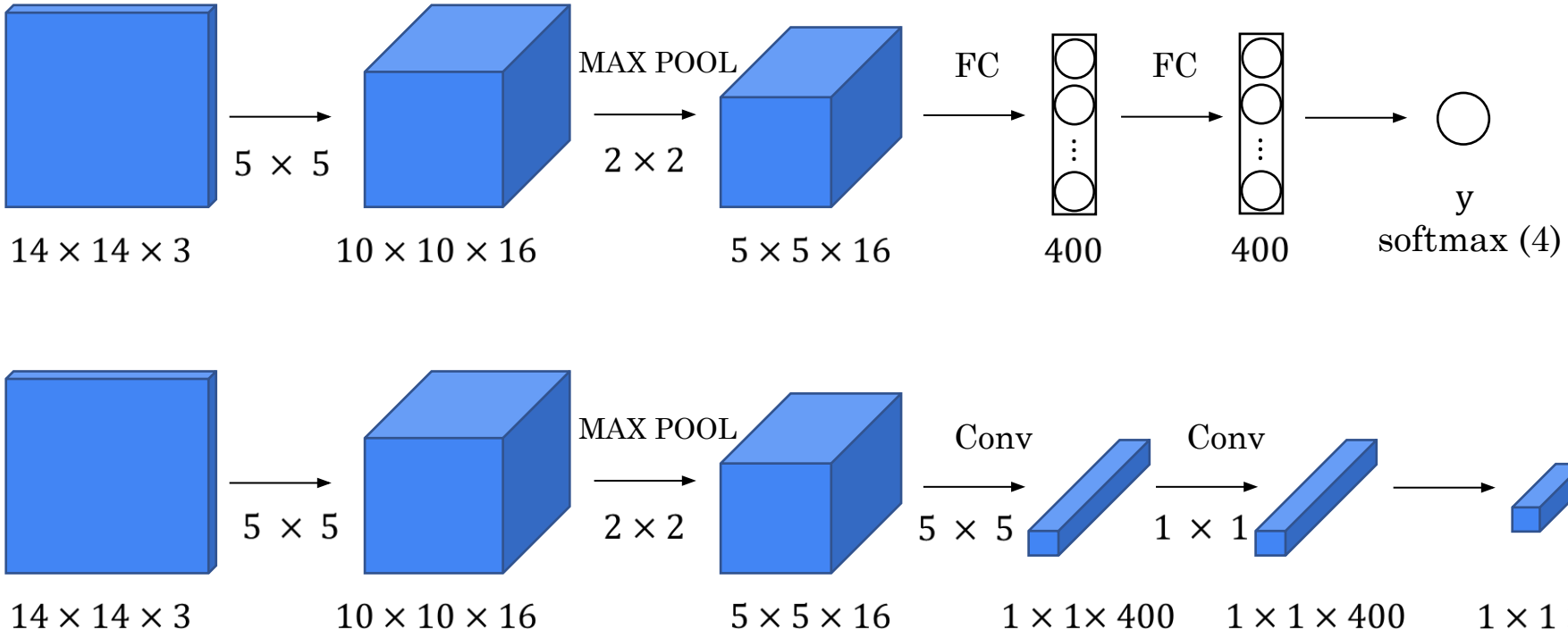
YOLO Detection

1. Basic Concepts:
    - Detection by sliding windows
    - Intersection over Union
    - Non-Max Suppression
    - Anchor Boxes
2. Final Yolo Architectures

YOLO Detection

1. Basic Concepts:
    - **Detection by sliding windows**
    - Intersection over Union
    - Non-Max Suppression
    - Anchor Boxes
2. Final Yolo Architectures

Turning FC layer into convolutional layers



Turning FC layer into convolutional layers

1st. FC Layer:
- Layer 5x5x16 is flattened resulting in 400 nodes, which are connected with another 400 nodes
- Each of the 400 nodes in the FC layer is a linear combination of the 5x5x16 volume followed by activation

Convolutional Layer 1x1x400:
- Each of the 400 filters is 5x5x16
- Each node of the convolutional layer is a linear combination of the 5x5x16 volume followed by activation

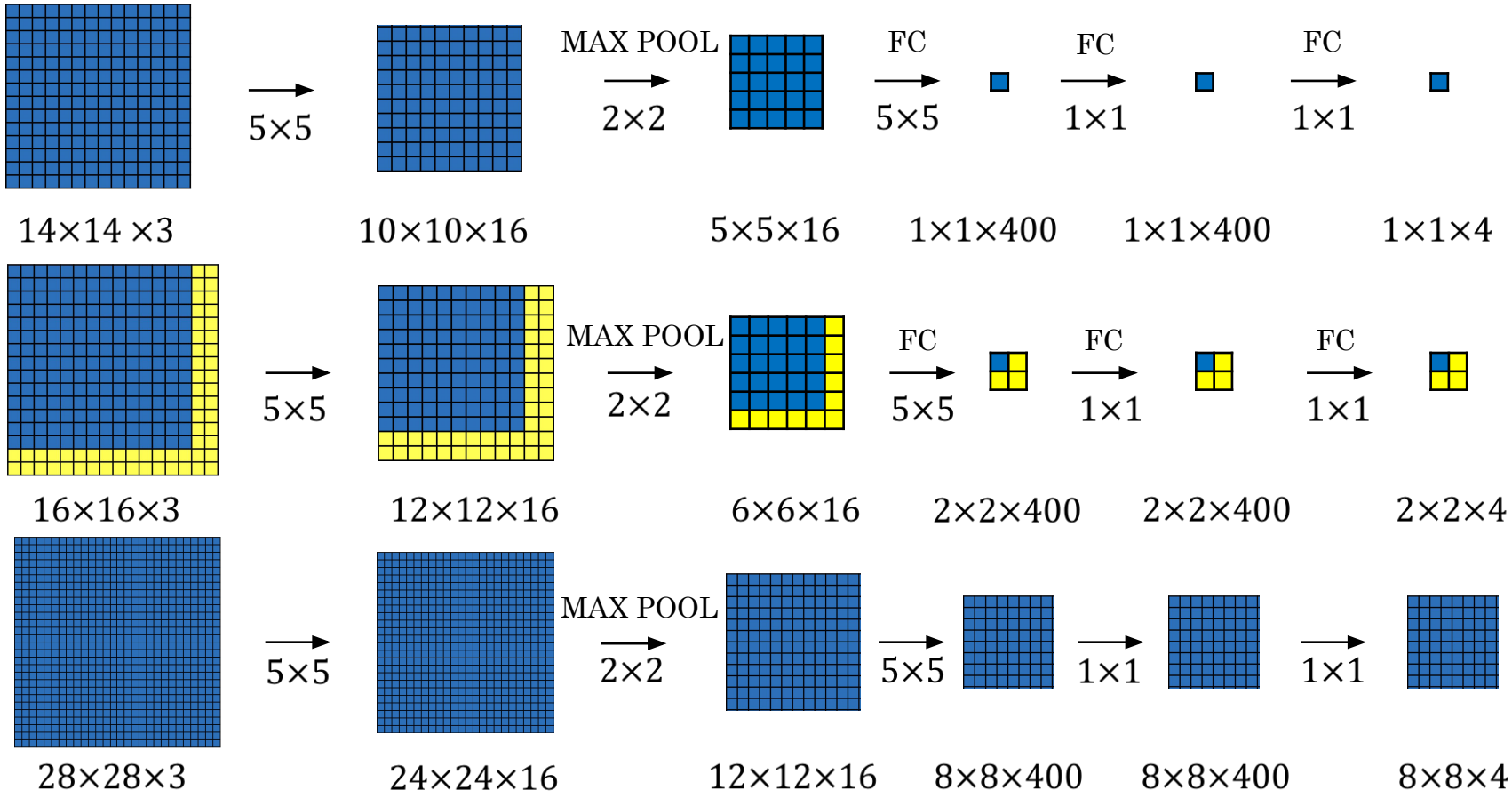
2nd. FC Layer:
- Each of the 400 nodes in the 2nd FC layer is a combination of the 400 nodes in the 1st FC followed by activation

Convolutional Layer 1x1x400:
- Each of the 400 filters is 1x1x400
- Each of the 400 nodes in the 2nd conv layer. 1x1x400 is a combination of the 400 nodes of the 1st. conv layer. Followed by activation

Last Layer:
- Combines previous layer activations into 4 nodes + softmax activation

Last Layer:
- Each of the 4 filters is 1x1x400
- Softmax activation

Convolution implementation of sliding windows

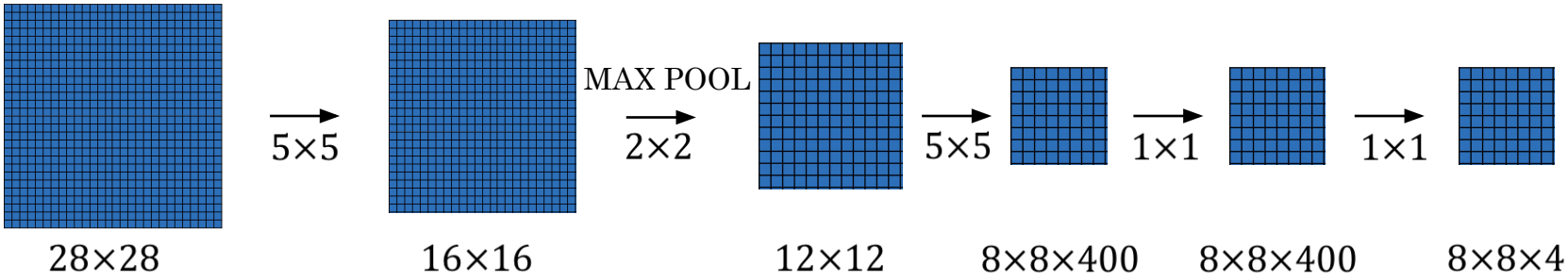


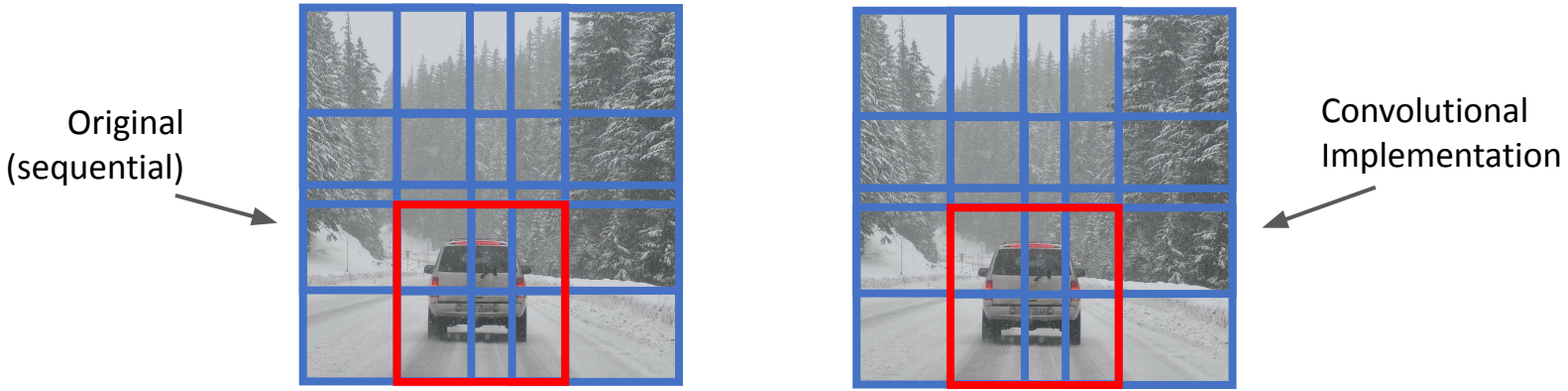
[Sermanet et al., 2014, OverFeat: Integrated recognition, localization and detection using convolutional networks]

Convolution implementation of sliding windows

- The example illustrate small numbers and only frontal side of tensors
- Sizes:
  - Training image: 14x14x3
  - Test image: 16x16x3
- Assume use of sliding windows with stride 2 → 4 windows
- Using sliding windows causes a lot of redundant computation
  - Result when passing test image over the network: 2x2x4
  - Each 1x1x4 volume corresponds to a window in the original image
- When test image is 28x28x3, result is a 8x8x4 volume
- The window stride in the original image is related to the 2x2 max-pooling

Convolution implementation of sliding windows





YOLO Detection

1. Basic Concepts:
    - Detection by sliding windows
    - **Intersection over Union**
    - Non-Max Suppression
    - Anchor Boxes
2. Final Yolo Architectures

Intersection over Union

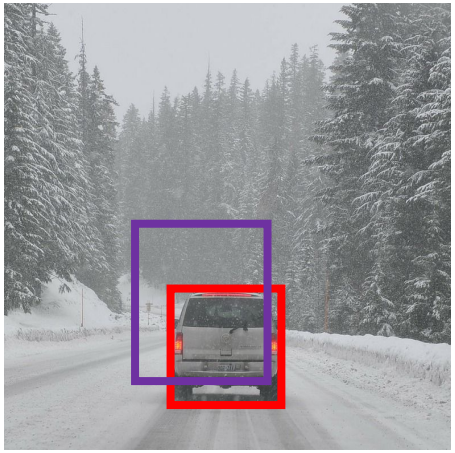
- How to evaluate whether the object detection algorithm is working well?
- IoU (intersection over union) metric: ratio between intersection and union
  - **0** → no overlap
  - **1** → identical bounding box
  - It is a special case of the Jaccard index

    Given two sets A and B

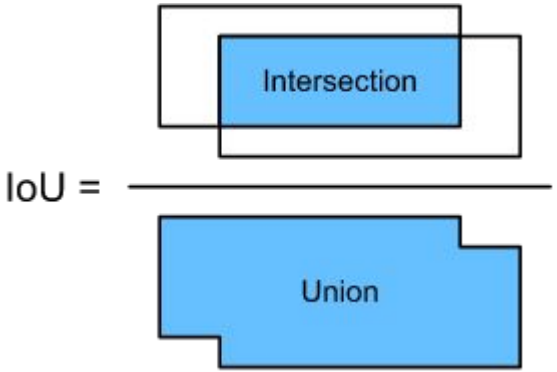
$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

- Also used as a component of certain detection algorithms

Evaluating the object position (predicted vs ground truth)



Intersection over Union (IoU)



“Correct” if IoU $\geq$ 0.5

More generally, IoU is a measure of overlap between two bounding boxes.

Evaluating the object position (predicted vs ground truth)

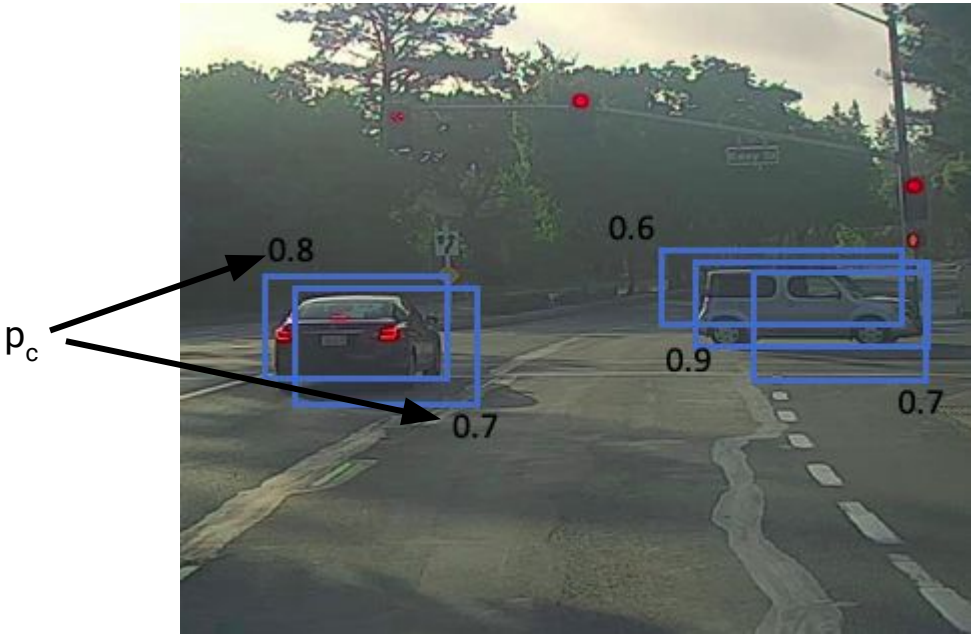
- Predicted bounding box: **purple**
- Correct bounding box: **red**
- Generally, returned region is correct if IoU ≥ 0.5
  - In some cases the threshold is 0.6 or 0.7
  - Rarely threshold is below 0.5
- Can be used to evaluate the quality of the returned bounding box
- More generally, it is a measure of overlap between regions

YOLO Detection

1. Basic Concepts:
    - Detection by sliding windows
    - Intersection over Union
    - **Non-Max Suppression**
    - Anchor Boxes
2. Final Yolo Architectures

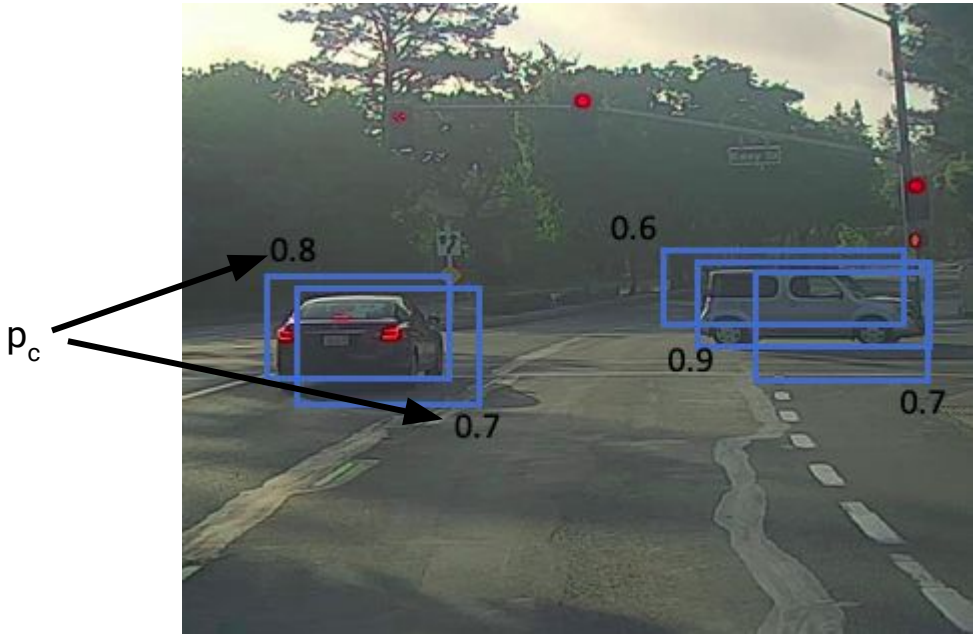
Problem with algorithms seen so far

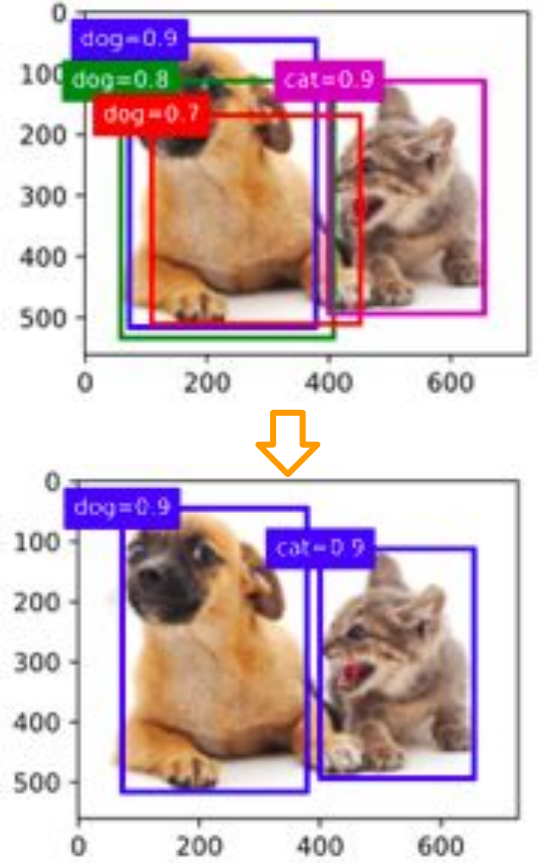
- Can find multiple bounding boxes referring to the same object



Problem with algorithms seen so far

- Can find multiple bounding boxes referring to the same object
- Non-max suppression allows you to detect the object once





Example of non-max suppression

- Non-max suppression aims to return only one box per object
- First, it compute p_c for all bounding boxes
  - Get the box with the biggest p_c
  - All boxes with high IoU are discarded
- Keep picking the bounding box with the highest p_c among the remaining ones, until each box has been selected or removed
- In the example, the two predictions correspond to the boxes with p_c = 0.9 e 0.8

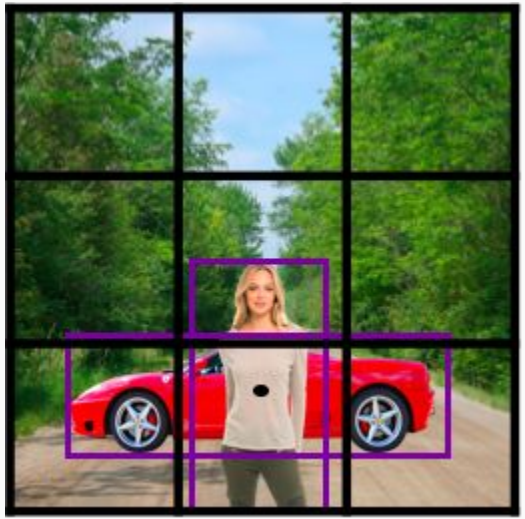
YOLO Detection

1. Basic Concepts:
    - Detection by sliding windows
    - Intersection over Union
    - Non-Max Suppression
    - **Anchor Boxes**
2. Final Yolo Architectures

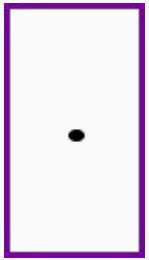
Anchor Boxes

- Object detection algorithms typically sample many regions of the input image and adjust the edges to return accurate bounding boxes.
- Different models may use different methods to sample regions
- We will see how to use anchor boxes for this
- Anchor boxes are bounding boxes of multiple sizes and aspect ratios

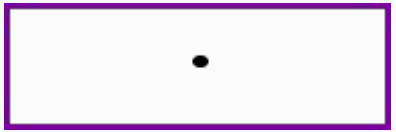
Overlapping Objects



Anchor box 1



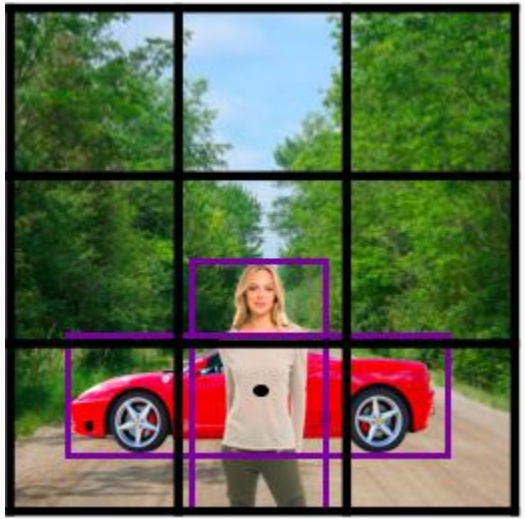
Anchor box 2



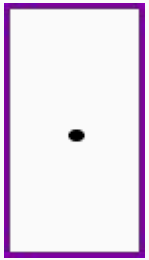
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

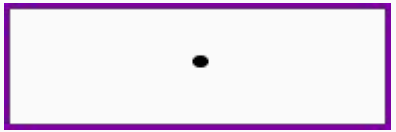
Overlapping Objects



Anchor box 1



Anchor box 2



$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ \vdots \\ c_3 \end{bmatrix}$$

Anchor box 1

Anchor box 2

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Overlapping Objects

- Cannot return two objects using the previously defined y vector
- Output: use anchor boxes with different formats
- We saw an example with 2 anchor boxes, but in practice ~5 to 10 are used.

  In our example:

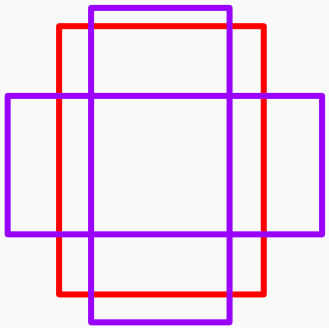
  - We use the first one to code the pedestrian
  - We use the second one to code the car

Anchor Box

Before (without anchor boxes):

- Each object in the training image was associated with the grid cell containing the object's midpoint

Output y:
3 x 3 x 8

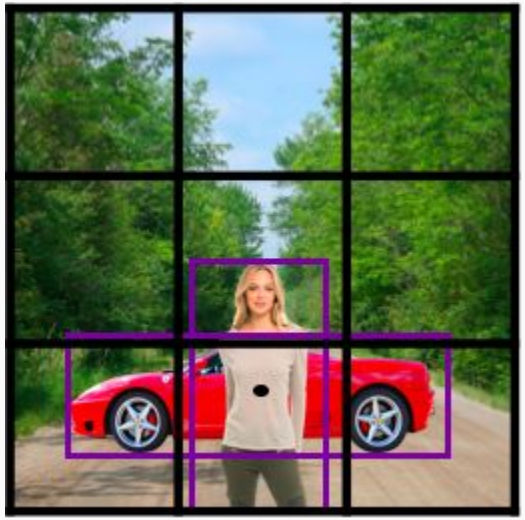


Now (with two anchor boxes per region):

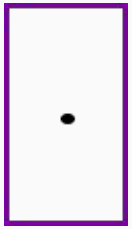
- Each region containing multiple anchor boxes is an example of training
- Each object in the training image, delimited by a bounding box, is associated with the anchor box with the highest IoU.

Output y:
3 x 3 x 16 ou
3 x 3 x 2 x 8

Example with anchor boxes



Anchor box 1



Anchor box 2

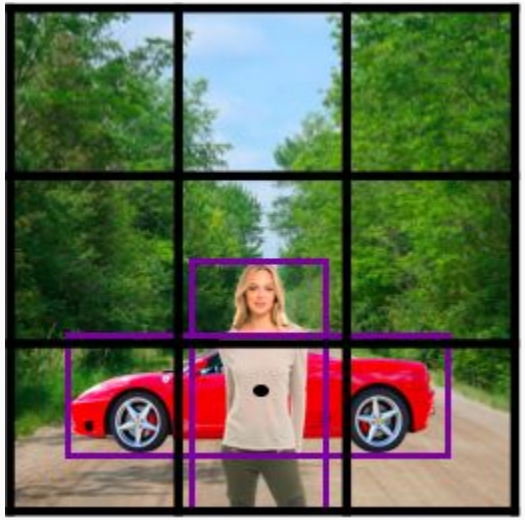
This image

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

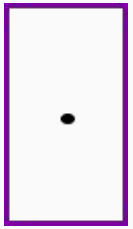
What if we have only a car?

$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Example with anchor boxes



Anchor box 1



Anchor box 2

This image

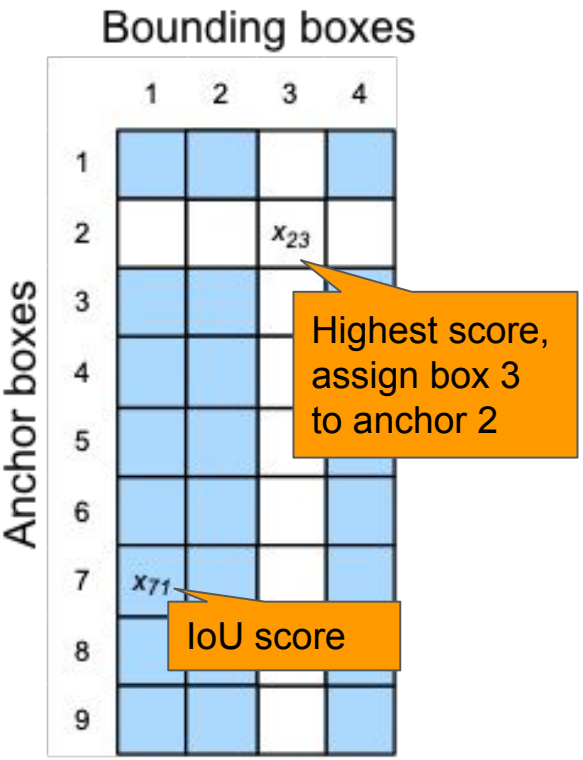
What if we have only a car?

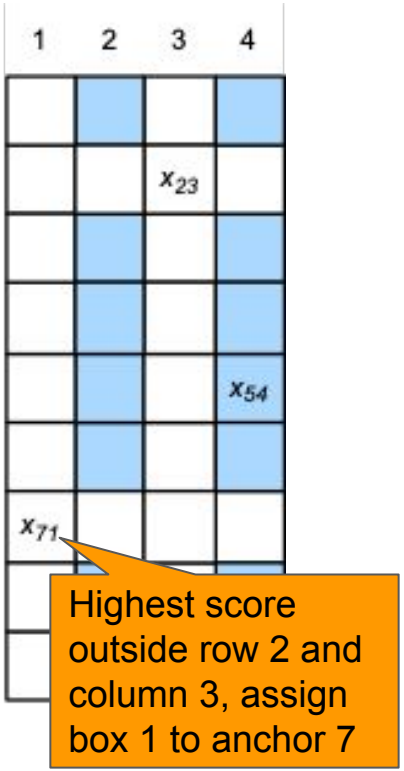
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \quad \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 1 \\ 0 \\ 0 \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

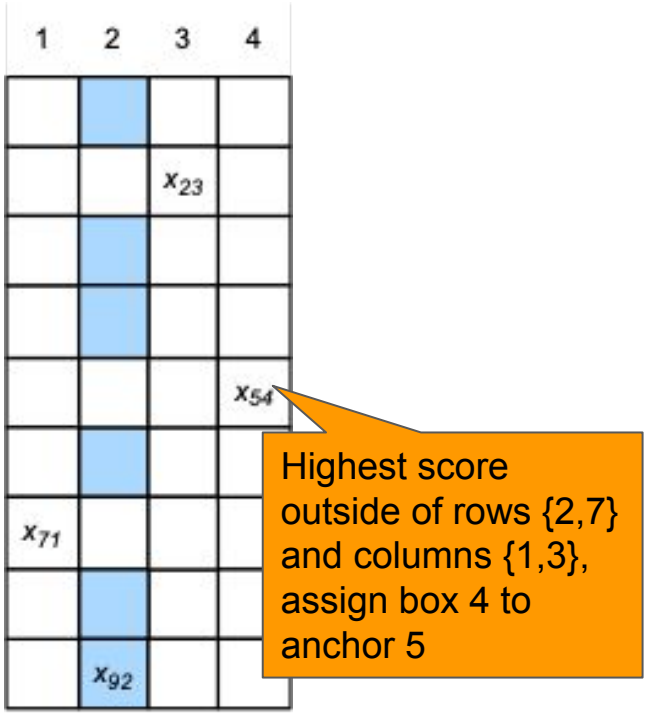
Example with anchor box

- Dimension 8 in 3x3x2x8 has to do with the number of classes
  - What if there were 6 classes?
- In the previous image, we had a car and a pedestrian If there was only one car, only the 2nd. half of the vector would be filled
- What if we had 2 anchor boxes, but 3 objects?
  - Hopefully it doesn't happen, otherwise, rule for breaking ties
- What if we had 2 objects associated with the same anchor box in a cell?
  - It wouldn't work perfectly either.
- In general, these problems do not occur with thinner grids.
- How to choose anchor boxes?
  - Manually, from 5 to 10
  - Using k-means algorithm to group objects according to shape and then choose representative anchor boxes

Assigning labels to anchor boxes

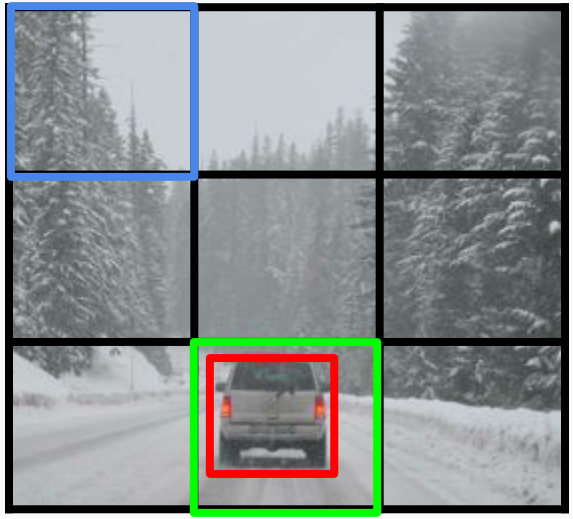






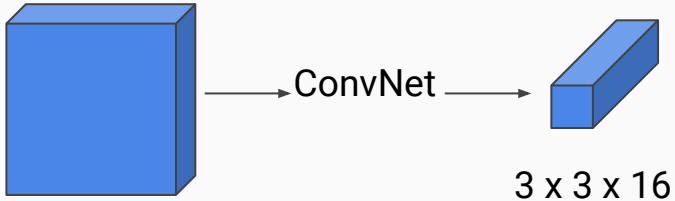
Putting it all together: YOLO Algorithm

Training Process



y é 3 x 3 x 2 x 8

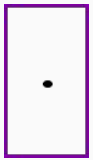
1 - pedestrian
2 - car
3 - motorcycle



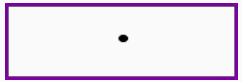
100 x 100 x 3

3 x 3 x 16

Anchor box 1



Anchor box 2

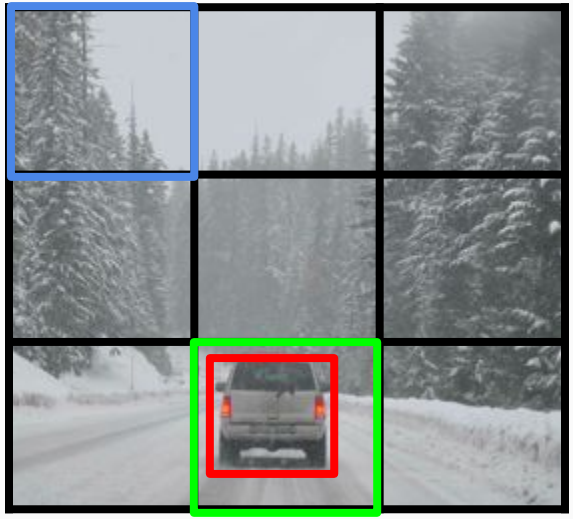


Cell 1 Cell 8

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

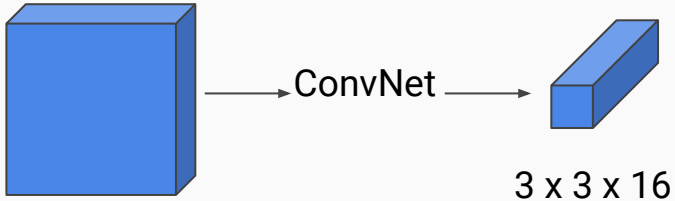
Training Process



y é 3 x 3 x 2 x 8

What if
... grid is smaller? 19 x 19 x 16
... 5 anchor boxes? 19 x 19 x 40

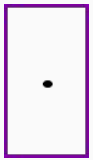
1 - pedestrian
2 - car
3 - motorcycle



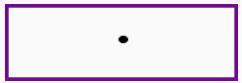
100 x 100 x 3

3 x 3 x 16

Anchor box 1



Anchor box 2



$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Cell 1

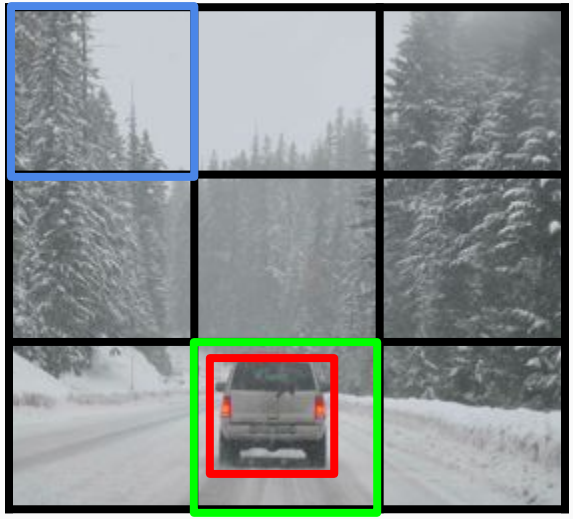
$$\begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$

Cell 8

$$\begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

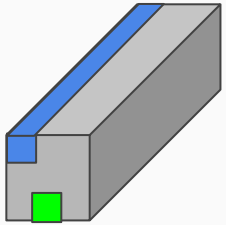
[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Prediction process



100 x 100 x 3

ConvNet



3 x 3 x 16

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

prediction 1

$$\begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$

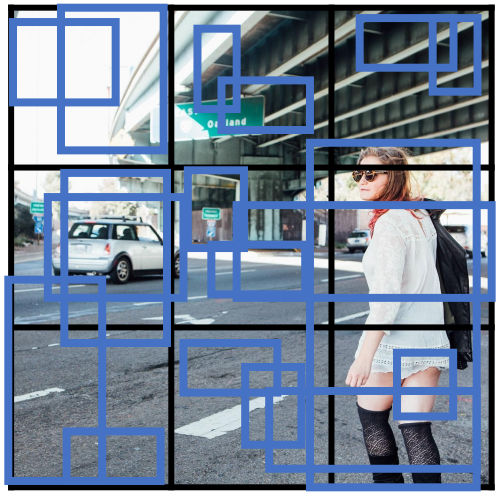
prediction 8

$$\begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

YOLO

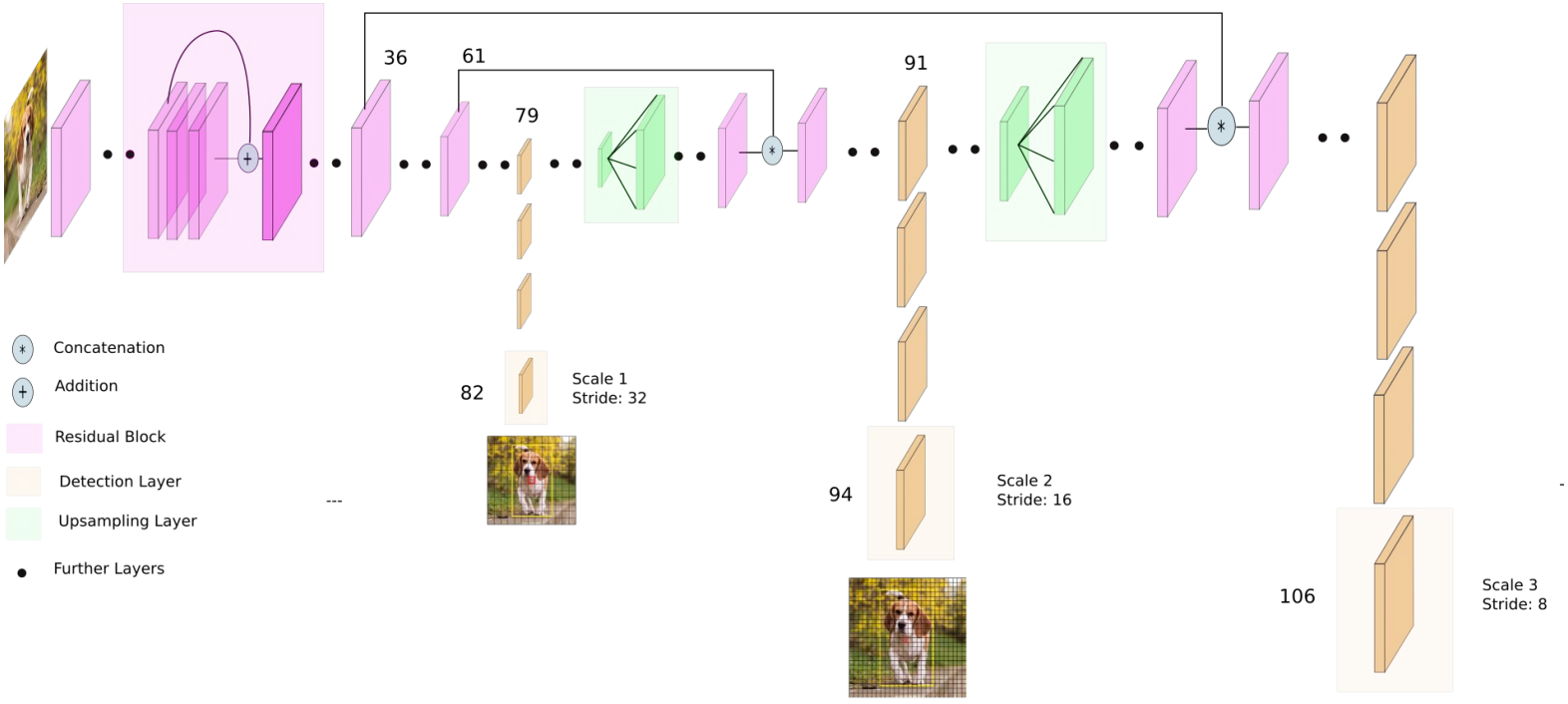
- In the previous example:
  - Training:
    - Cell 1 have no object
    - Cell 8 has 1 car, with higher IoU assigned to anchor box 1
  - Prediction:
    - Cell 1: expected that $p_c = 0$, i.e. other inputs doesn't matter
    - Cell 8: anchor box 2 is expected to match the car

Returning output after non-max suppression



- For each cell, return 2 bounding boxes
- Remove low probability predictions
- For each class (pedestrian, car, motorcycle) use non-max suppression to generate the final predictions

YOLO Architecture



YOLO v3 network Architecture

QUIZ about YOLO Detection

In object detection using non-maximum suppression (NMS), suppose we have detected bounding boxes with their corresponding confidence scores and IoU values as follows:

Box 1: Confidence Score = 0.85, IoU with Box 2 = 0.6

Box 2: Confidence Score = 0.75, IoU with Box 1 = 0.6, IoU with Box 4 = 0.7

Box 3: Confidence Score = 0.90, IoU with Box 4 = 0.4

Box 4: Confidence Score = 0.80, IoU with Box 2 = 0.7, IoU with Box 3 = 0.4, IoU with Box 5 = 0.8

Box 5: Confidence Score = 0.95, IoU with Box 4 = 0.8

https://www.menti.com/al4g8cdbmyix

Results:
https://www.mentimeter.com/app/presentation/al4p6syc37n4guj2t9e8z97p4yfkpwf8